ИТМО

Отчет по лабораторной работе №2

по методам оптимизации

Продвинутые методы безусловной оптимизации

Невыпуклая функция: функция Била

Модель регрессии: Ridge

Модель классификации: Probit

Трек: Потеря ортогональности в NLCG и формула Hager--Zhang

Участники команды:
1) Болотин Тимофей
2) Калачин Алексей
3) Ржевин Андрей

2026

Содержание

Введение . . . 3
Цель работы . . . 3
Используемое ПО и оборудование . . . 3
Характеристики ПК каждого из студентов . . . 3
Распределение задач между участниками . . . 4
Теоретические сведения и формулы . . . 4
Общая постановка задачи . . . 4
Критерий остановки . . . 4
Линейный поиск . . . 4
Используемые методы оптимизации . . . 5
Формулы для выбранных ML-моделей . . . 5
Реализация и проверка корректности . . . 7
Проверка градиента . . . 7
Проверка произведения гессиана на вектор . . . 7
Вывод . . . 7
Эксперименты . . . 7
Эксперимент 2.2: Метод сопряжённых градиентов (CG) — сравнение с градиентным спуском . . . 7
Цель и постановка . . . 7
Настройки . . . 8
Данные . . . 8
Результаты . . . 8
Выводы . . . 9
Эксперимент 2.3: Метод L-BFGS — выбор размера истории . . . 9
Цель и постановка . . . 9
Настройки . . . 9
Результаты . . . 10
Вопросы . . . 17
Выводы . . . 17
Эксперимент 2.4: Сравнение методов на реальных ML-задачах . . . 18
Постановка . . . 18
Настройки . . . 18
Данные . . . 18
Результаты . . . 19
Ответы на вопросы . . . 24
Выводы . . . 25
Эксперимент 2.5: Микропрофилирование вычислительных затрат . . . 25
Постановка задачи . . . 25
Используемые данные и модели . . . 25
Результаты . . . 26
Ответы на вопросы . . . 28
Выводы . . . 29
Эксперимент 2.6: Оптимизационная точность против качества предсказания . . . 29

Постановка задачи 29
Данные, модели и настройки 29
Результаты 30
Ответы на вопросы 32
Выводы 35
Эксперимент 2.7 (Индивидуальный трек): Потеря ортогональности в нелинейном методе сопряжённых градиентов и формула Hager–Zhang 35
Цель и постановка 35
Настройки 36
Данные 36
Результаты 36
Анализ результатов 37
Выводы 38

Введение

В этой лабораторной работе рассматриваются продвинутые методы безусловной оптимизации для гладких задач. Основное внимание уделяется методу сопряжённых градиентов, нелинейному методу сопряжённых градиентов, усечённому методу Ньютона и методу L-BFGS.

Эксперименты проводятся на квадратичных задачах и на реальных задачах машинного обучения. Для регрессии используется Ridge-регрессия на датасетах `housing_scale` и `pyrim`, для классификации — Probit-регрессия на датасетах `w8a` и `colon-cancer`. Также проводится индивидуальное исследование потери ортогональности в нелинейном методе сопряжённых градиентов и сравнение формул Полака–Рибьера и Hager–Zhang.

Цель работы

Цель работы — реализовать и исследовать продвинутые методы безусловной оптимизации: метод сопряжённых градиентов, нелинейный CG, усечённый метод Ньютона и L-BFGS. Нужно сравнить их поведение на квадратичных задачах и ML-оракулах, изучить влияние размера истории в L-BFGS, провести микропрофилирование вычислительных затрат и проверить, как оптимизационная точность связана с качеством модели на тестовой выборке.

В индивидуальном треке исследуется потеря ортогональности соседних градиентов в нелинейном методе сопряжённых градиентов. Дополнительно сравниваются формулы PR и Hager–Zhang, а также анализируется влияние рестарта Пауэлла.

Используемое ПО и оборудование

Лабораторная работа выполнялась с использованием:

- Язык/среда: Python 3.14.3
- Библиотеки: `numpy`, `scipy`, `sklearn`, `matplotlib`
- ОС: Windows 11 и Arch Linux

Характеристики ПК каждого из студентов

Болотин Тимофей:

- CPU: Intel(R) Core(TM) Ultra 5 125H, 1200 MHz, 14 ядер; RAM: 32 GB;
- ОС: Windows 11.

Калачин Алексей:

- CPU: AMD Ryzen AI 7 H 350, 16 потоков, до 5.09 GHz; RAM: 32 GB;
- ОС: Arch Linux 6.19.9.

Ржевин Андрей:

- CPU: Intel(R) Core(TM) Ultra 5 125H, 3600 MHz, 14 ядер; RAM: 16 GB;
- ОС: Windows 11.

Распределение задач между участниками

Участник	Выполненная часть
Ржевин Андрей	Эксперимент 2.2 и индивидуальный трек 2.7.
Болотин Тимофей	Эксперименты 2.3 и 2.4.
Калачин Алексей	Эксперименты 2.5 и 2.6, а также сборка общих частей отчета.

Таблица 1. Распределение задач между участниками команды.

Теоретические сведения и формулы

Общая постановка задачи

Рассматривается задача гладкой безусловной оптимизации

$$\min_{x \in \mathbb{R}^n} f(x),$$

где $f : \mathbb{R}^n \to \mathbb{R}$ — достаточно гладкая функция. Методы оптимизации строят последовательность точек

$$x_{k+1} = x_k + \alpha_k d_k,$$

где d_k — направление поиска, а α_k — длина шага. Для методов спуска требуется, чтобы направление удовлетворяло условию

$$\langle \nabla f(x_k), d_k \rangle < 0.$$

В задачах машинного обучения минимизируется регуляризованный эмпирический риск:

$$f(x) = \frac{1}{m} \sum_{i=1}^{m} L(b_i, \langle a_i, x \rangle) + \frac{\lambda}{2} \, \|x\|_2^2,$$

где $A \in \mathbb{R}^{m \times n}$ — матрица признаков, $b \in \mathbb{R}^m$ — вектор ответов, $x \in \mathbb{R}^n$ — вектор параметров модели, $\lambda > 0$ — коэффициент L2-регуляризации.

Критерий остановки

Во всех основных экспериментах использовался относительный критерий остановки по квадрату нормы градиента:

$$\|\nabla f(x_k)\|_2^2 \le \varepsilon \, \|\nabla f(x_0)\|_2^2.$$

Такой критерий не зависит от масштаба функции и позволяет сравнивать методы на разных задачах. Дополнительно задавалось максимальное число итераций, чтобы фиксировать случаи, когда метод не достиг заданной точности.

Линейный поиск

Для выбора длины шага в основных экспериментах использовался линейный поиск по условиям Вульфа. Для функции одной переменной

$$\varphi_{k(\alpha)} = f(x_k + \alpha d_k)$$

условия Вульфа контролируют достаточное убывание функции и изменение производной вдоль направления поиска. В методах Ньютона, HFN и L-BFGS линейный поиск начинался с $\alpha = 1$, так как единичный шаг важен для быстрой локальной сходимости.

Если поиск Вульфа не находил подходящий шаг, использовался backtracking по условию Армихо. Также в коде поддерживались константный шаг и точный шаг для квадратичных задач.

Используемые методы оптимизации

В работе использовались следующие методы.

- **Метод сопряжённых градиентов (CG).** Используется для решения квадратичных задач и систем линейных уравнений с симметричной положительно определённой матрицей. Метод не требует явного обращения матрицы и использует только произведения матрицы на вектор.
- **Нелинейный метод сопряжённых градиентов (NLCG PR).** Для неквадратичных функций направление обновляется по правилу

$$d_{k+1} = -\nabla f(x_{k+1}) + \beta_k d_k.$$

  В качестве основной формулы использовалась формула Полака–Рибьера:

$$\beta_k^{\mathrm{PR}} = \frac{\nabla f(x_{k+1})^T (\nabla f(x_{k+1}) - \nabla f(x_k))}{\|\nabla f(x_k)\|_2^2}.$$

- **Усечённый метод Ньютона (HFN).** На каждой внешней итерации ньютоновское направление находится приближённо из системы

$$\nabla^2 f(x_k) d_k = -\nabla f(x_k).$$

  Система решается внутренним методом сопряжённых градиентов. Явно строить гессиан не требуется: используется операция $\nabla^2 f(x_k) v$.
- **L-BFGS.** Это квазиньютоновский метод, который приближает действие обратного гессиана по истории последних пар

$$s_k = x_{k+1} - x_k, \quad y_k = \nabla f(x_{k+1}) - \nabla f(x_k).$$

  В отличие от полного BFGS, метод хранит только L последних пар векторов, поэтому требует памяти порядка $O(nL)$.
- **Градиентный спуск (GD).** Использовался как базовый метод первого порядка:

$$x_{k+1} = x_k - \alpha_k \nabla f(x_k).$$

- **Полный метод Ньютона.** Использовался для сравнения на задачах, где размерность позволяла явно построить гессиан и решить систему с ним.

Формулы для выбранных ML-моделей

Пусть $A \in \mathbb{R}^{m \times n}$ — матрица признаков, $b \in \mathbb{R}^m$ — вектор ответов, $x \in \mathbb{R}^n$ — вектор параметров, $\lambda > 0$ — коэффициент L2-регуляризации.

Ridge-регрессия.

Целевая функция:

$$f(x) = \frac{1}{2m}\ \|Ax - b\|_2^2 + \frac{\lambda}{2}\ \|x\|_2^2.$$

Градиент:

$$\nabla f(x) = \frac{1}{m} A^T (Ax - b) + \lambda x.$$

Гессиан:

$$\nabla^2 f(x) = \frac{1}{m} A^T A + \lambda I.$$

Произведение гессиана на вектор:

$$\nabla^2 f(x) v = \frac{1}{m} A^T A v + \lambda v.$$

Probit-классификация.

Пусть $b_i \in \{-1, 1\}$, а

$$t = b \cdot (Ax),$$

где $\cdot$ обозначает поэлементное произведение. Целевая функция:

$$f(x) = -\frac{1}{m} 1^T \log \Phi(t) + \frac{\lambda}{2}\ \|x\|_2^2,$$

где Φ — функция распределения стандартного нормального закона.

Обозначим

$$r(t) = \frac{\varphi(t)}{\Phi(t)},$$

где φ — плотность стандартного нормального распределения. Тогда градиент имеет вид

$$\nabla f(x) = -\frac{1}{m} A^T (b \cdot r(t)) + \lambda x.$$

Для гессиана вводится

$$s(t) = r(t) \cdot (t + r(t)).$$

Тогда

$$\nabla^2 f(x) = \frac{1}{m} A^T\ \mathrm{diag}(s(t)) A + \lambda I.$$

Произведение гессиана на вектор:

$$\nabla^2 f(x)v = \frac{1}{m} A^T (s(t) \cdot (Av)) + \lambda v.$$

Реализация и проверка корректности

Проверка градиента

Корректность градиентов проверялась с помощью конечных разностей. Для каждой координаты использовалась аппроксимация

$$[\nabla f(x)]_i \approx \frac{f(x + \varepsilon_1 e_i) - f(x)}{\varepsilon_1},$$

где e_i — базисный вектор, а ε_1 выбиралось малым. Проверка выполнялась для Ridge- и Probit-оракулов на тестовых данных. Аналитический градиент сравнивался с численным, после чего считалась относительная ошибка.

Проверка произведения гессиана на вектор

Для усечённого метода Ньютона в ЛР2 важно проверить операцию $\nabla^2 f(x)v$, так как именно она используется во внутреннем методе сопряжённых градиентов. Проверка выполнялась через конечные разности:

$$\left[\nabla^2 f(x)v\right]_i \approx \frac{f(x + \varepsilon_2 v + \varepsilon_2 e_i) - f(x + \varepsilon_2 v) - f(x + \varepsilon_2 e_i) + f(x)}{\varepsilon_2^2}.$$

Проверка проводилась для обоих ML-оракулов. Аналитическое произведение гессиана на вектор совпадало с разностной аппроксимацией с малой относительной ошибкой, поэтому реализацию `hess_vec` можно считать корректной.

Вывод

Реализованные оракулы корректно вычисляют значение функции, градиент и произведение гессиана на вектор. Это позволяет использовать их в методах первого порядка, L-BFGS и усечённом методе Ньютона. Проверка конечными разностями подтверждает, что дальнейшие эксперименты сравнивают поведение методов оптимизации, а не ошибки реализации производных.

Эксперименты

Эксперимент 2.2: Метод сопряжённых градиентов (CG) — сравнение с градиентным спуском

Цель и постановка

Исследуется зависимость числа итераций метода сопряжённых градиентов и градиентного спуска от числа обусловленности κ и размерности пространства n при решении квадратичной задачи.

Настройки

Использовались размерности $n \in \{10, 100, 1000\}$ и значения κ по логарифмической сетке от 10^0 до 10^4. Для каждой пары (n, κ) проводилось 5 запусков с усреднением результата. Начальное приближение: $x_0 = 0$. Критерий остановки: tolerance = 1e-8. Для градиентного спуска использовался метод Wolfe, max_iter = 20000. Предобусловливание и регуляризация не применялись.

Данные

Матрица A — диагональная симметричная положительно определённая, с диагональными элементами из $[1, \kappa]$, при этом минимальное значение фиксировалось как 1, максимальное — как κ. Вектор b генерировался случайно из нормального распределения.

Результаты

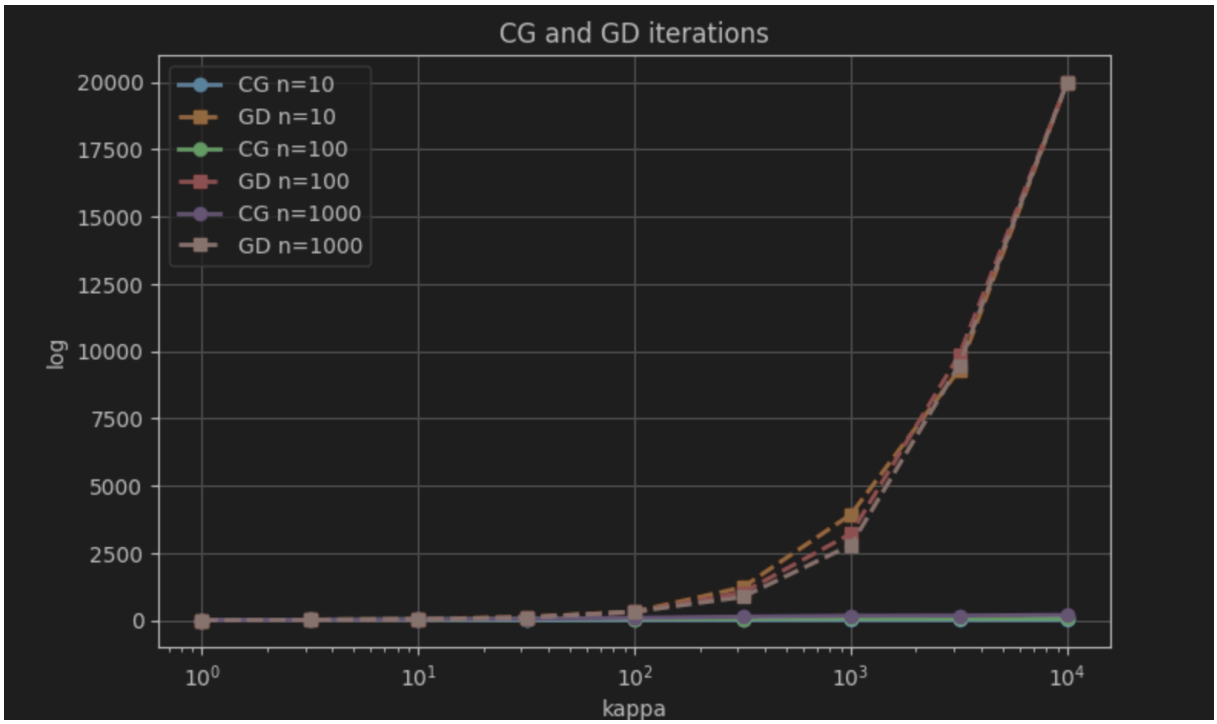
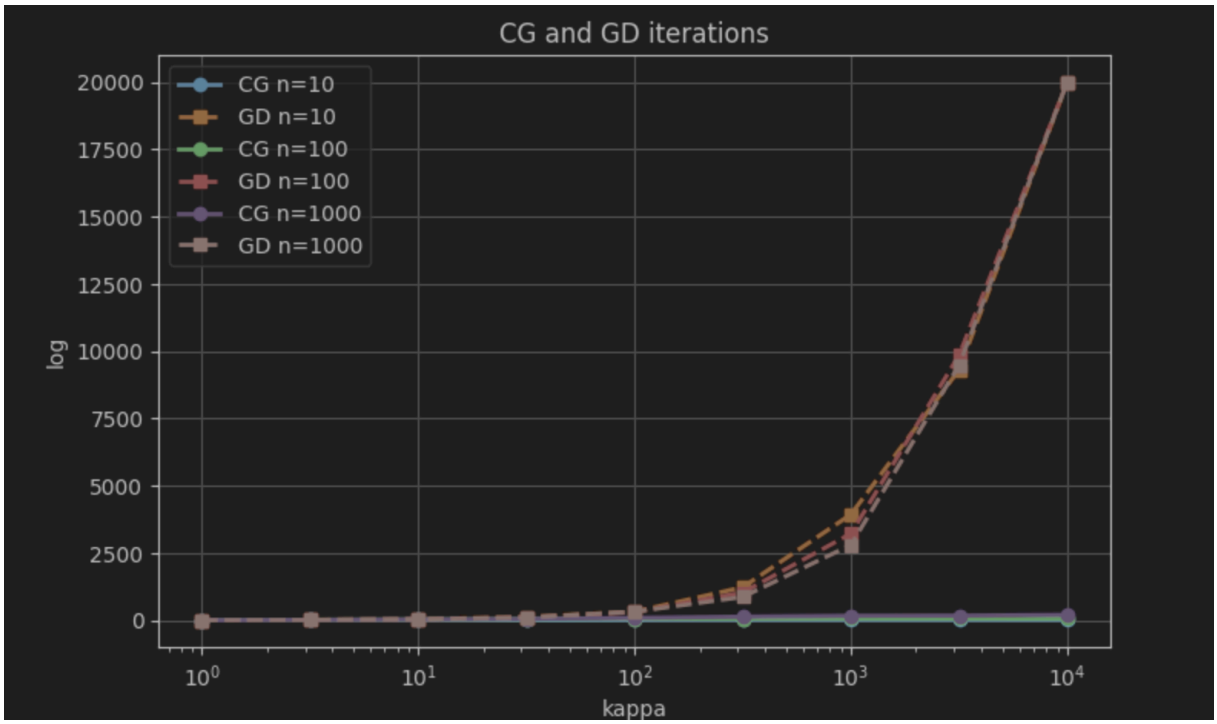


Рис. 1. Сравнение числа итераций CG и GD в зависимости от числа обусловленности для n = 10, 100, 1000



По графику видно, что число итераций GD быстро растёт с увеличением κ и при больших значениях доходит до ограничения max_iter = 20000. Метод CG требует значительно меньше итераций и растёт по κ заметно медленнее. Влияние размерности на CG есть, но оно слабее, чем влияние обусловленности на GD.

Выводы

Метод сопряжённых градиентов оказался существенно эффективнее градиентного спуска на квадратичных задачах. Он достигает заданной точности за гораздо меньшее число итераций, особенно при больших числах обусловленности.

Эксперимент 2.3: Метод L-BFGS — выбор размера истории

Цель и постановка

В этом эксперименте исследуется, как размер истории L в методе L-BFGS влияет на скорость сходимости. Сравнивались значения $L \in \{0, 1, 5, 10, 50, 100\}$. Случай $L = 0$ соответствует градиентному спуску с тем же линейным поиском, так как информация о предыдущих шагах не используется.

Эксперимент проводился на ML-оракулах нашего варианта: Ridge-регрессия для датасетов `housing_scale`, `pyrim` и Probit-классификация для датасетов `w8a`, `colon-cancer`. Дополнительно была рассмотрена плохо обусловленная квадратичная задача для иллюстрации влияния обусловленности.

Настройки

Во всех ML-экспериментах использовались стандартные параметры:

- начальная точка: $x_0 = 0$;
- коэффициент регуляризации: $\lambda = \frac{1}{m}$;
- линейный поиск: Wolfe;

- критерий остановки: относительная норма градиента;
- размеры истории: $L \in \{0, 1, 5, 10, 50, 100\}$.

Память L-BFGS без учета оракула имеет порядок $O(nL)$, так как хранятся L пар векторов s_k и y_k. Сложность двухцикловой рекурсии на одной итерации также равна $O(nL)$.

Результаты

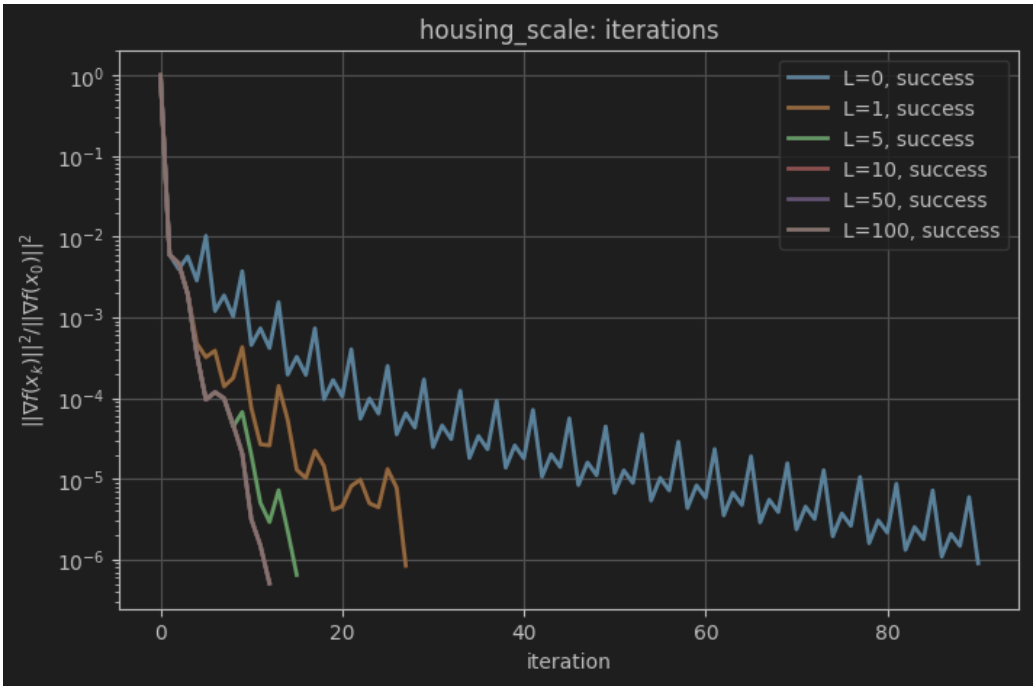


Рис. 2. Сходимость L-BFGS на `housing_scale` по числу итераций.

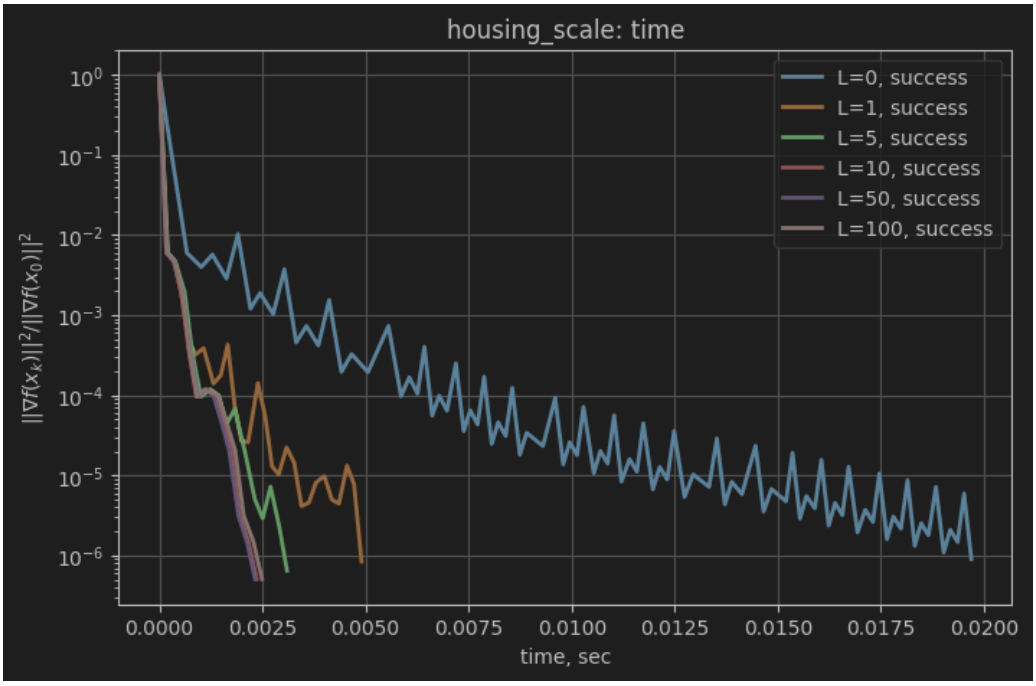


Рис. 3. Сходимость L-BFGS на `housing_scale` по времени.

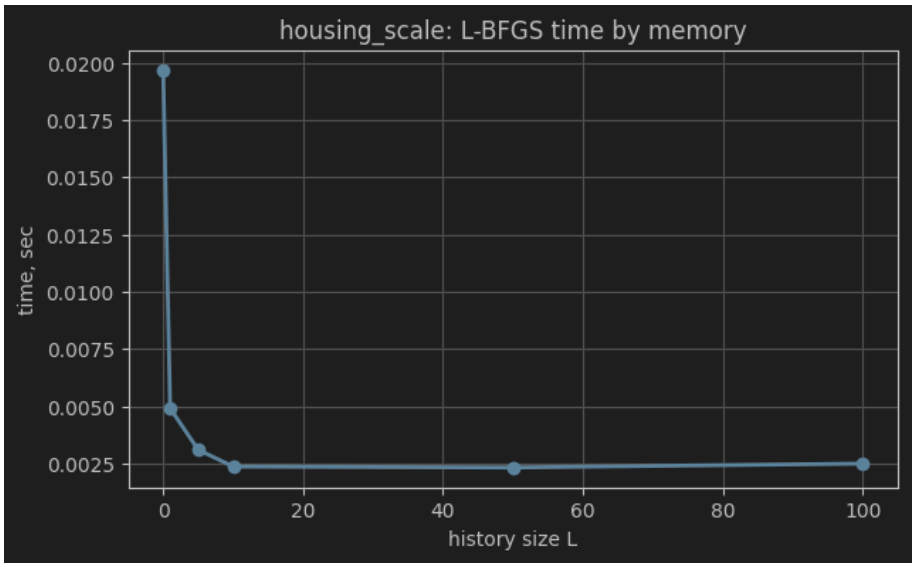


Рис. 4. Итоговое время работы на `housing_scale` в зависимости от размера истории L.

Для `housing_scale` видно, что даже небольшая история резко ускоряет сходимость по сравнению с $L = 0$. После $L = 5$–10 дальнейшее увеличение истории почти не дает выигрыша.

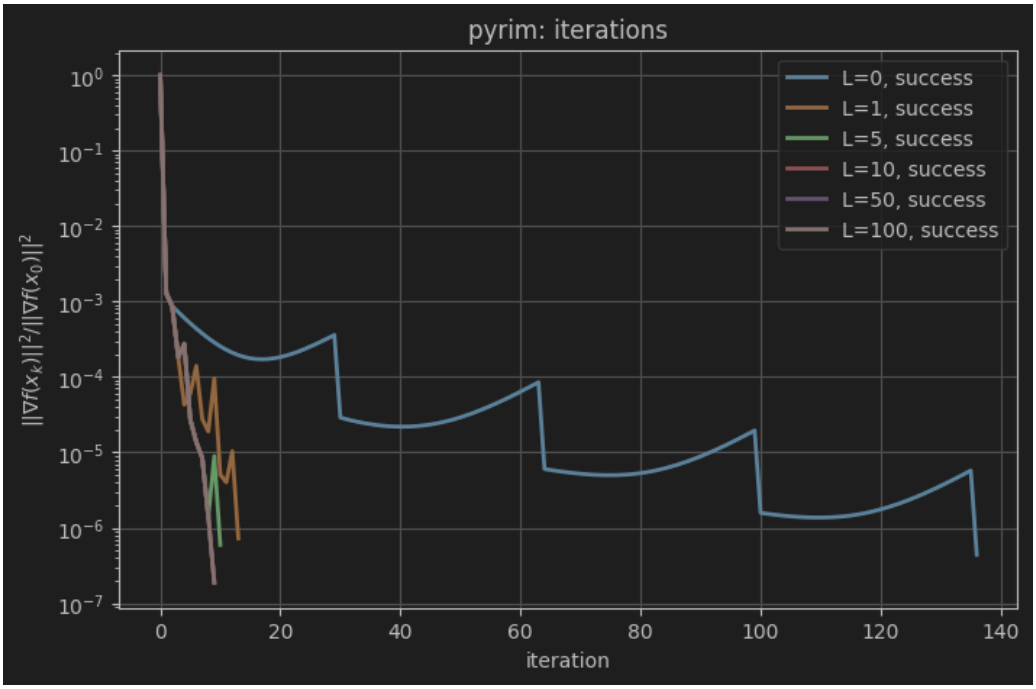


Рис. 5. Сходимость L-BFGS на `pyrim` по числу итераций.

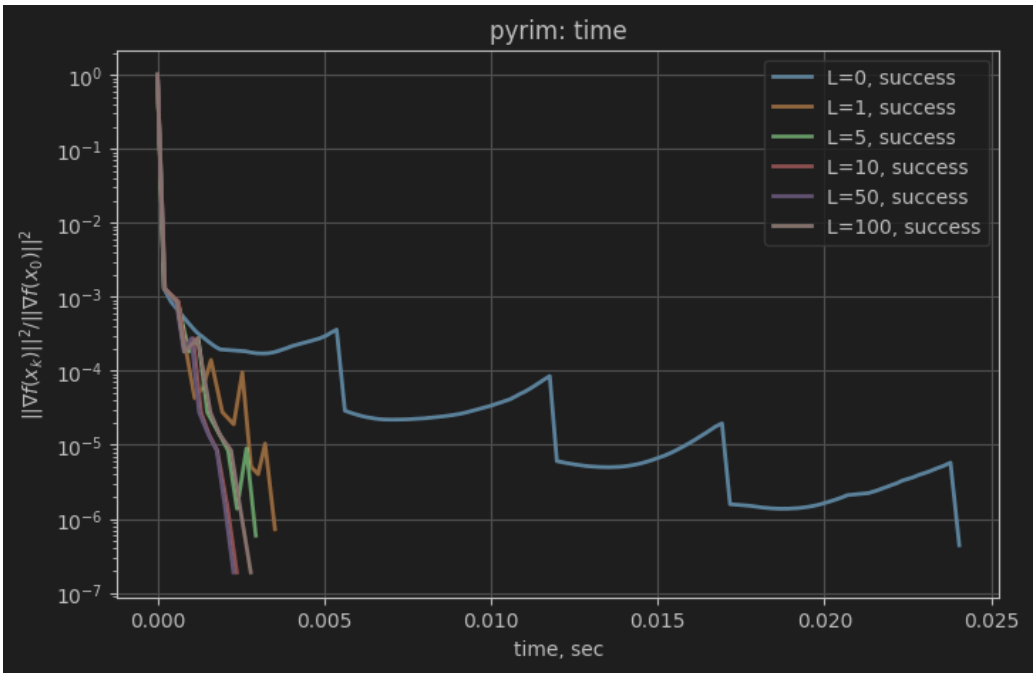


Рис. 6. Сходимость L-BFGS на `pyrim` по времени.

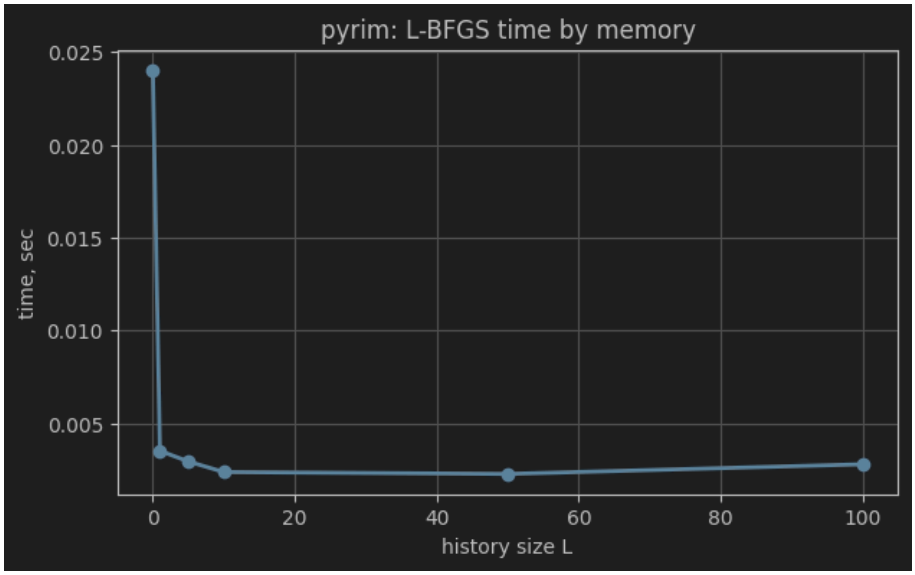


Рис. 7. Итоговое время работы на `pyrim` в зависимости от размера истории L.

На `pyrim` картина аналогична: $L = 0$ требует намного больше итераций, а варианты с $L \geq 5$ сходятся существенно быстрее. Слишком большое L уже почти не улучшает сходимость.

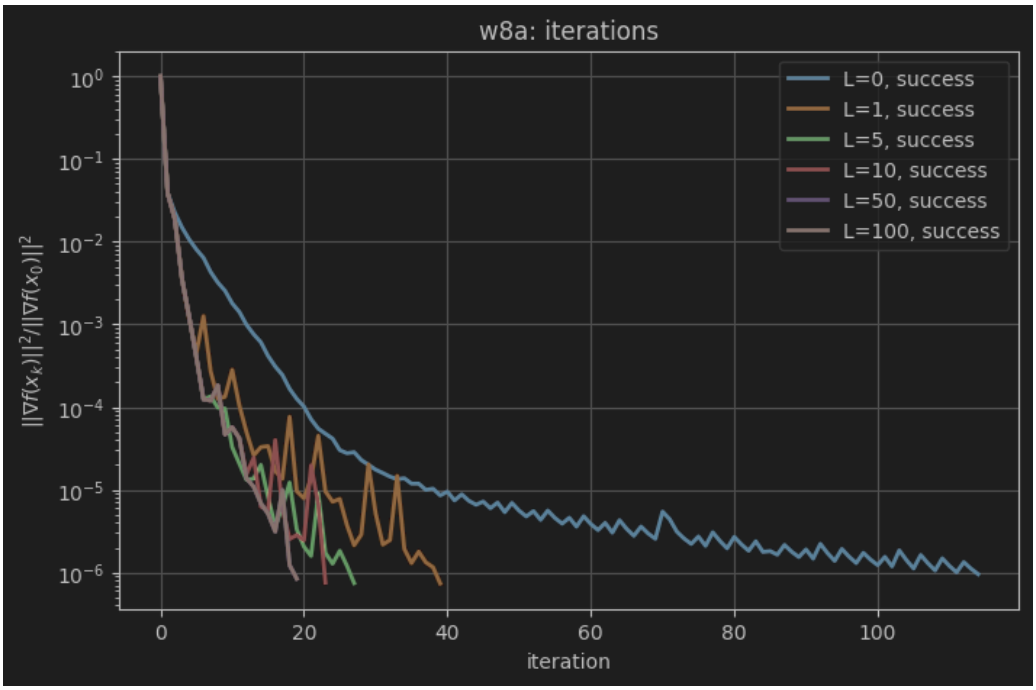


Рис. 8. Сходимость L-BFGS на w8a по числу итераций.

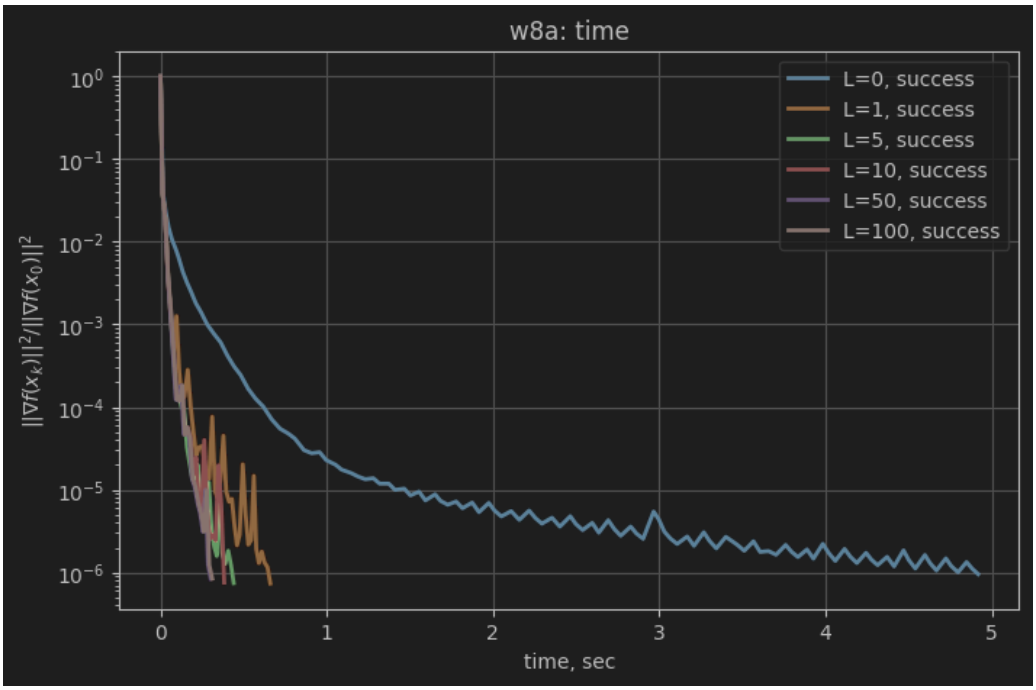


Рис. 9. Сходимость L-BFGS на w8a по времени.

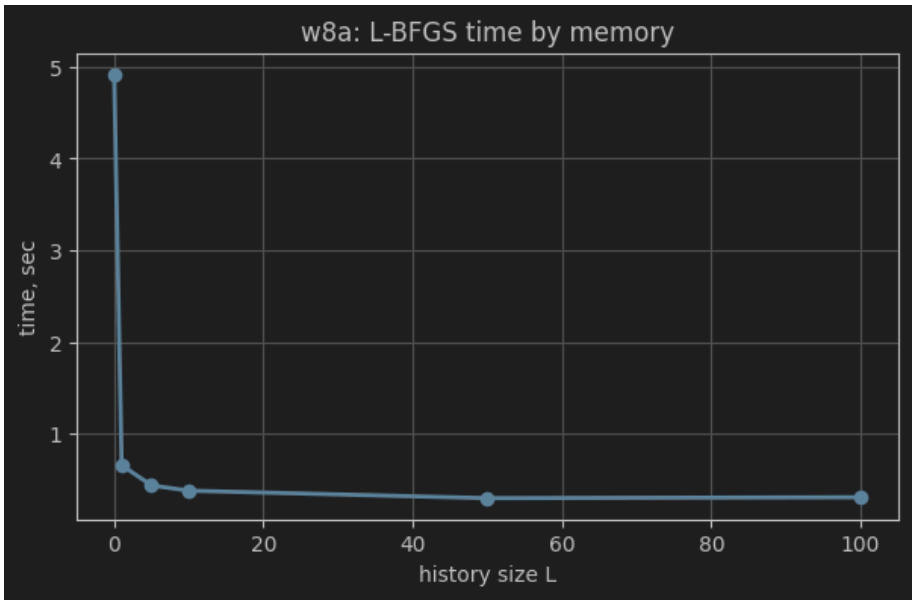


Рис. 10. Итоговое время работы на `w8a` в зависимости от размера истории L.

На большом датасете `w8a` учет истории особенно заметно ускоряет метод. По времени лучшими оказываются значения порядка $L = 10$–50, при этом $L = 0$ работает значительно дольше.

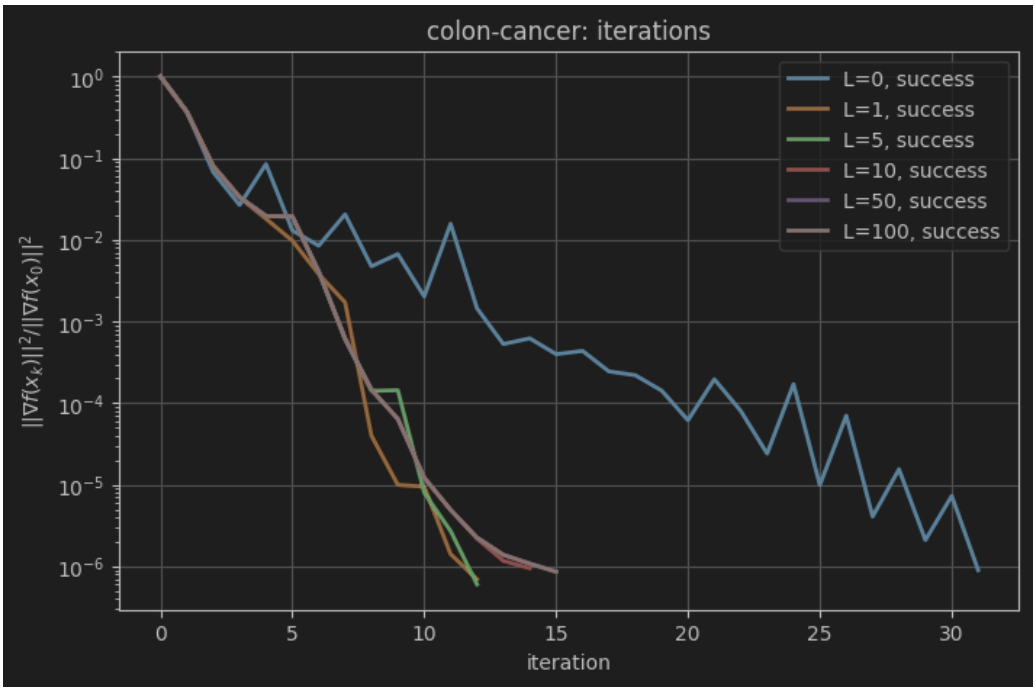


Рис. 11. Сходимость L-BFGS на `colon-cancer` по числу итераций.

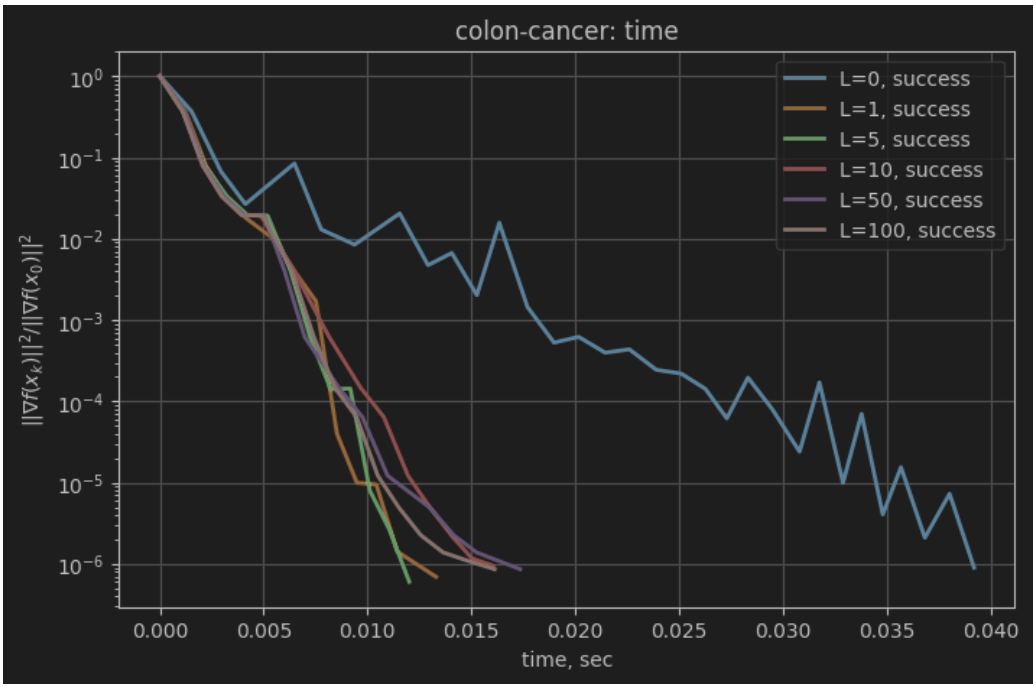


Рис. 12. Сходимость L-BFGS на `colon-cancer` по времени.

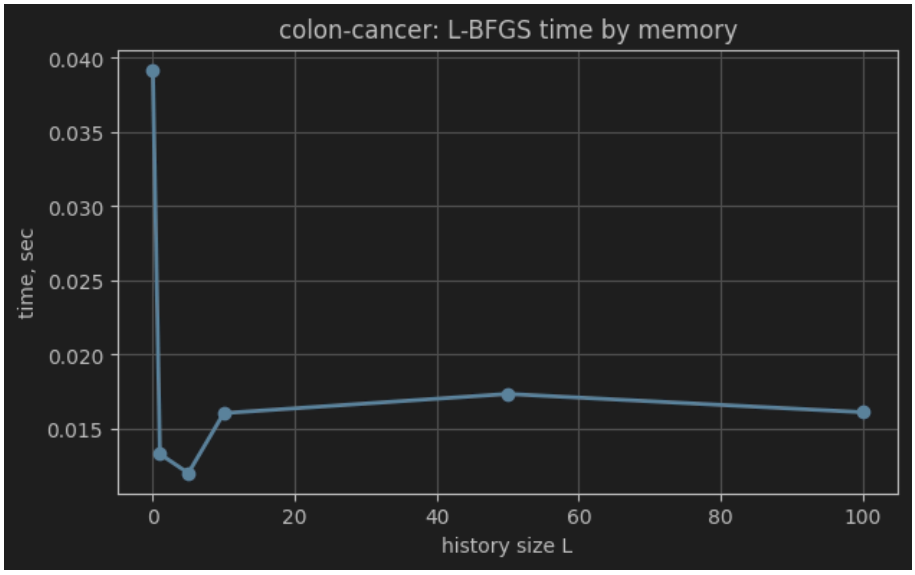


Рис. 13. Итоговое время работы на `colon-cancer` в зависимости от размера истории L.

На `colon-cancer` увеличение L тоже уменьшает число итераций по сравнению с $L = 0$, но слишком большая история уже замедляет метод по времени. Лучшее значение на этом датасете находится около $L = 5$.

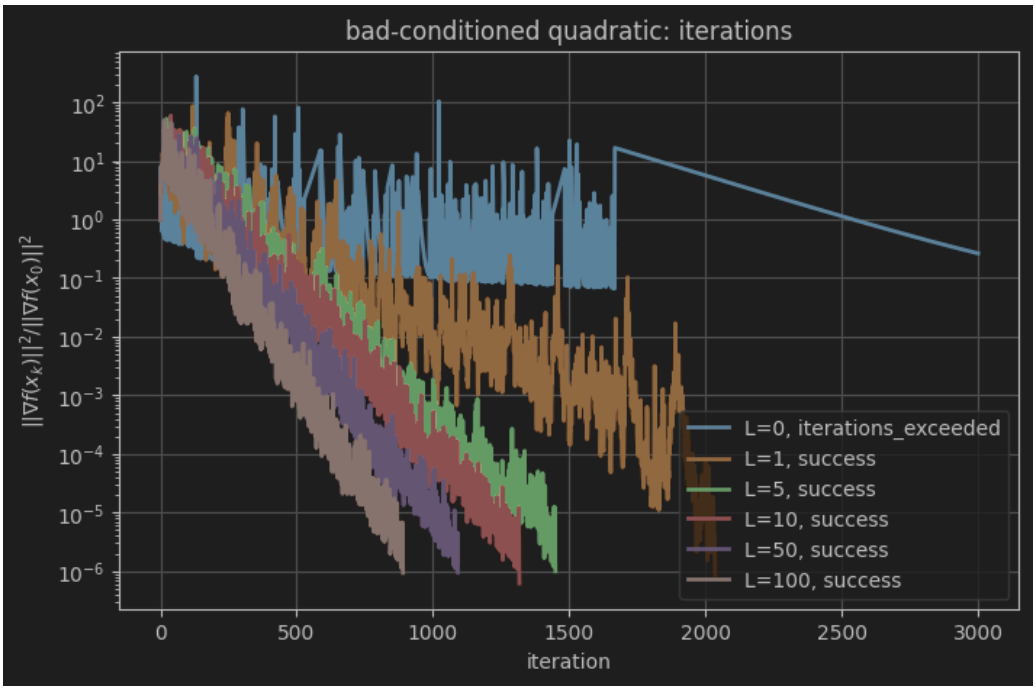


Рис. 14. Сходимость на плохо обусловленной квадратичной функции по числу итераций.

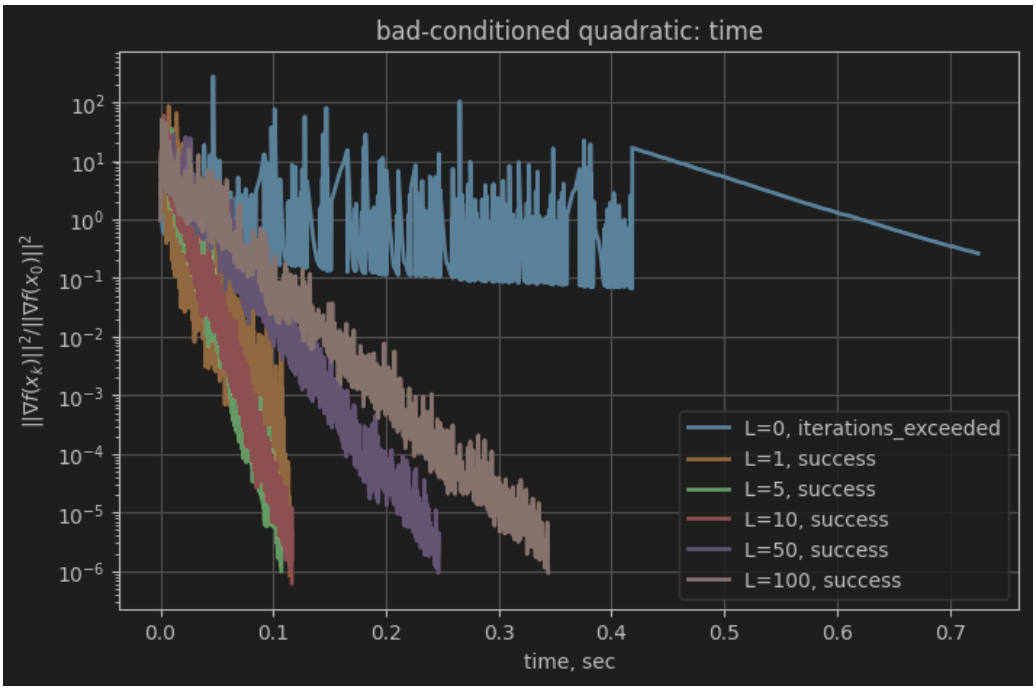


Рис. 15. Сходимость на плохо обусловленной квадратичной функции по времени.

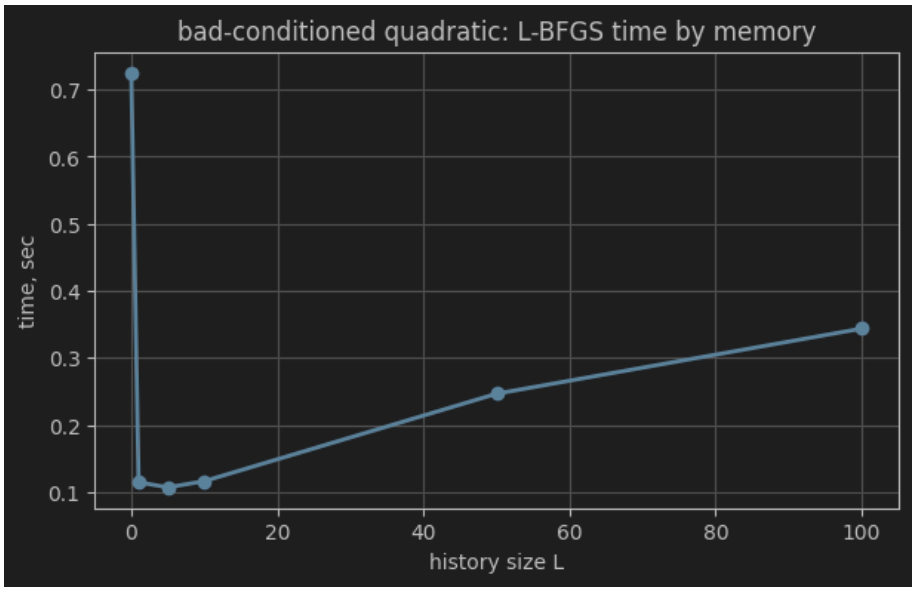


Рис. 16. Итоговое время работы на плохо обусловленной квадратичной функции в зависимости от размера истории L.

На плохо обусловленной квадратичной функции видно, что $L = 0$ не достигает сходимости за заданное число итераций. При $L \geq 1$ метод сходится, а значения $L = 5$ и $L = 10$ дают наиболее удачный баланс между числом итераций и временем.

Вопросы

1. На большинстве графиков после $L \approx 5$–10 увеличение истории почти не сокращает число итераций. Это значение намного меньше размерности задач, поэтому хранить большую историю обычно не требуется.

2. Оптимальное значение зависит от датасета: для `housing_scale`, `pyrim` и `colon-cancer` достаточно $L = 5$–10, а для `w8a` по времени лучше выглядит область $L = 10$–50. Слишком большое L может замедлять метод, потому что каждая итерация становится дороже из-за двухцикловой рекурсии.

3. На плохо обусловленной квадратичной функции история становится важнее: $L = 0$ не сходится за заданный лимит, а L-BFGS с ненулевой историей достигает успеха. При этом слишком большая история также не обязательна: $L = 5$–10 уже дает хороший результат.

4. Даже $L = 1$ и особенно $L = 5$ сильно ускоряют сходимость по сравнению с $L = 0$. Это видно на всех ML-датасетах: градиентный спуск убывает медленнее, а L-BFGS быстро использует информацию о кривизне из предыдущих шагов.

Выводы

Эксперимент показывает, что L-BFGS существенно выигрывает у градиентного спуска уже при небольшой истории. На практике нет смысла брать слишком большой L: после некоторого плато выигрыш по итерациям почти исчезает, а стоимость одной итерации продолжает расти. Для наших данных разумным выбором является $L = 5$–10, а для большого датасета `w8a` допустимо использовать немного большую историю.

Эксперимент 2.4: Сравнение методов на реальных ML-задачах

Постановка

В этом эксперименте сравниваются методы NLCG PR, HFN, L-BFGS, GD и полный Newton на ML-оракулах нашего варианта. Для L-BFGS использовался размер истории $L = 10$. GD и полный Newton добавлены для сравнения с методами из первой лабораторной работы.

Рассматривались две модели:

- Ridge-регрессия на датасетах `housing_scale`, `pyrim`;
- Probit-классификация на датасетах `w8a`, `colon-cancer`.

Настройки

Во всех экспериментах использовались:

- начальная точка: $x_0 = 0$;
- коэффициент регуляризации: $\lambda = \frac{1}{m}$;
- линейный поиск: Wolfe;
- критерий остановки: относительная норма градиента;
- `tolerance = 1e-6`;
- `max_iter = 500`;
- для L-BFGS: $L = 10$.

Полный метод Ньютона запускался только для задач с размерностью не больше 2500, так как он требует явного плотного гессиана.

Данные

Датасет	Задача	m	n
`housing_scale`	Ridge	506	13
`pyrim`	Ridge	74	27
`w8a`	Probit	49749	300
`colon-cancer`	Probit	62	2000

Таблица 2. Датасеты, использованные в эксперименте 2.4.

Результаты

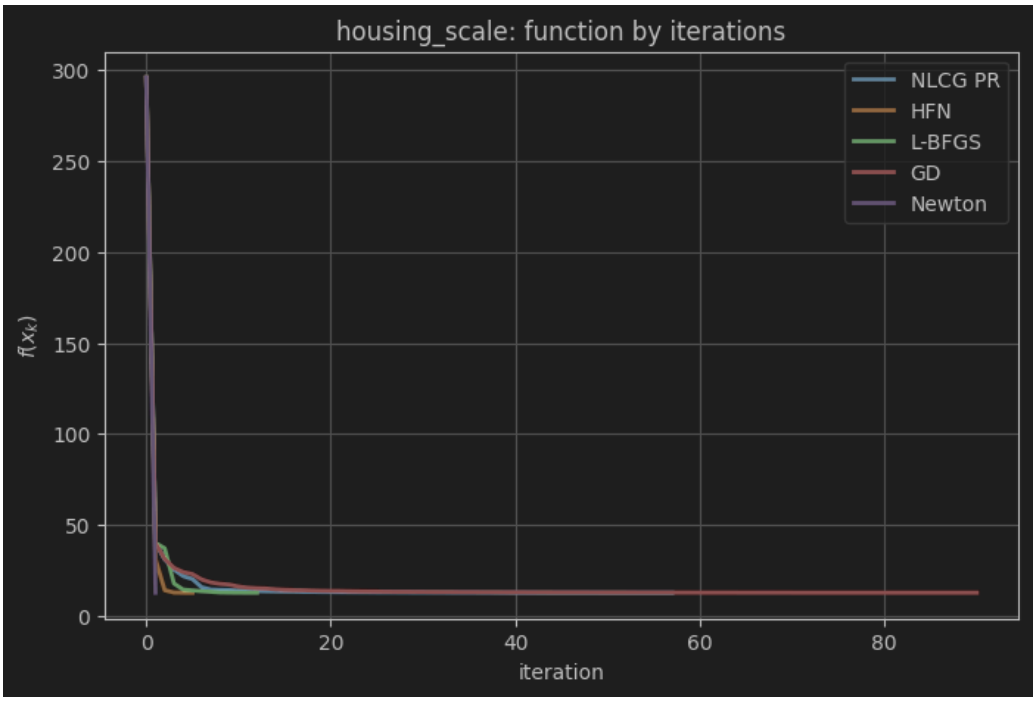


Рис. 17. `housing_scale`: значение функции от номера итерации.

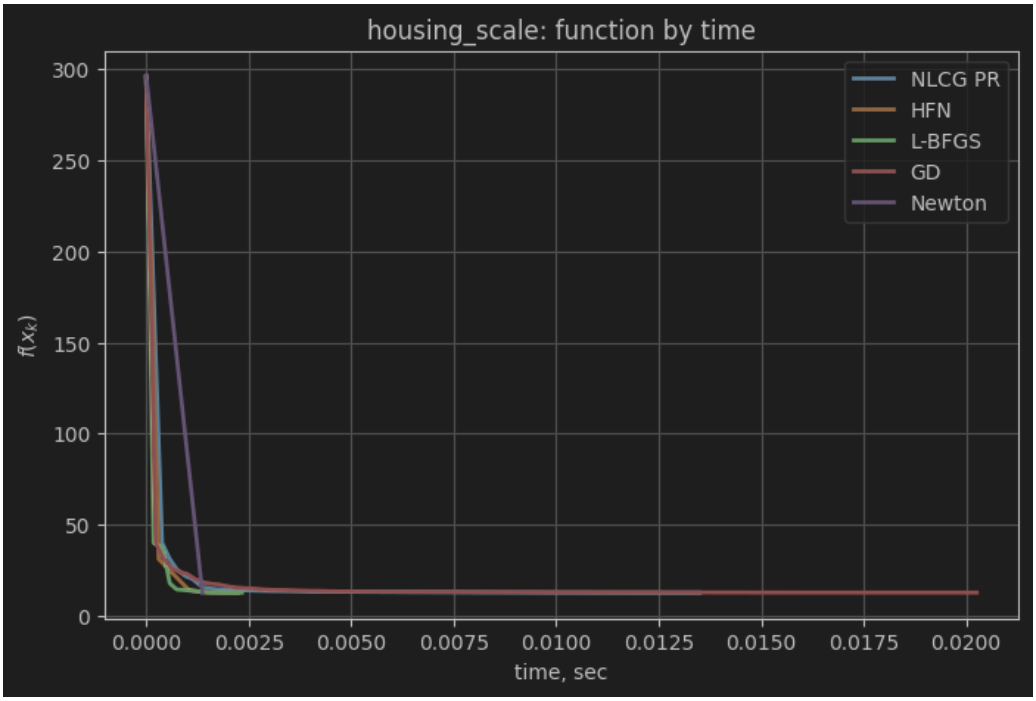


Рис. 18. `housing_scale`: значение функции от времени.

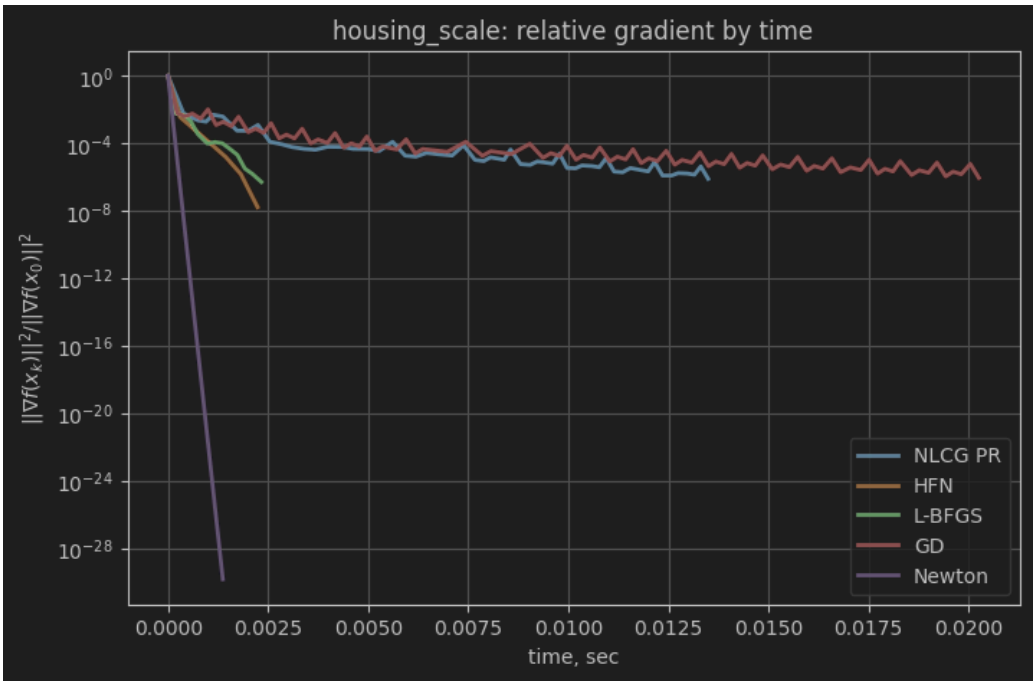


Рис. 19. `housing_scale`: относительный квадрат нормы градиента от времени.

На `housing_scale` полный Newton быстрее всех уменьшает норму градиента и достигает очень малой невязки за малое число итераций. Среди продвинутых методов HFN и L-BFGS сходятся быстрее, чем GD и NLCG PR.

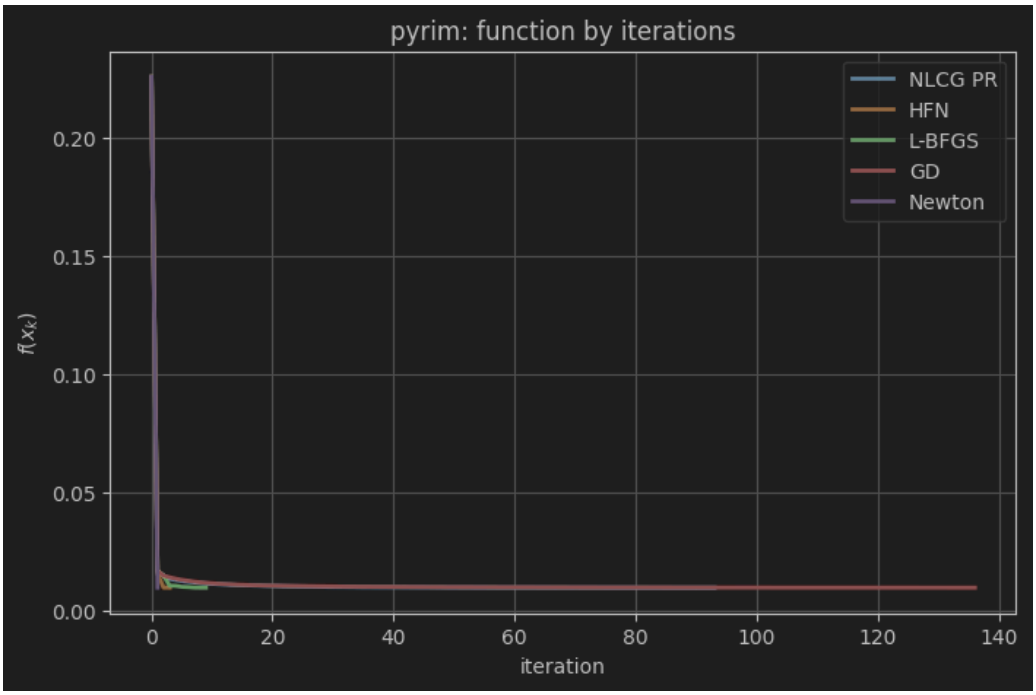


Рис. 20. `pyrim`: значение функции от номера итерации.

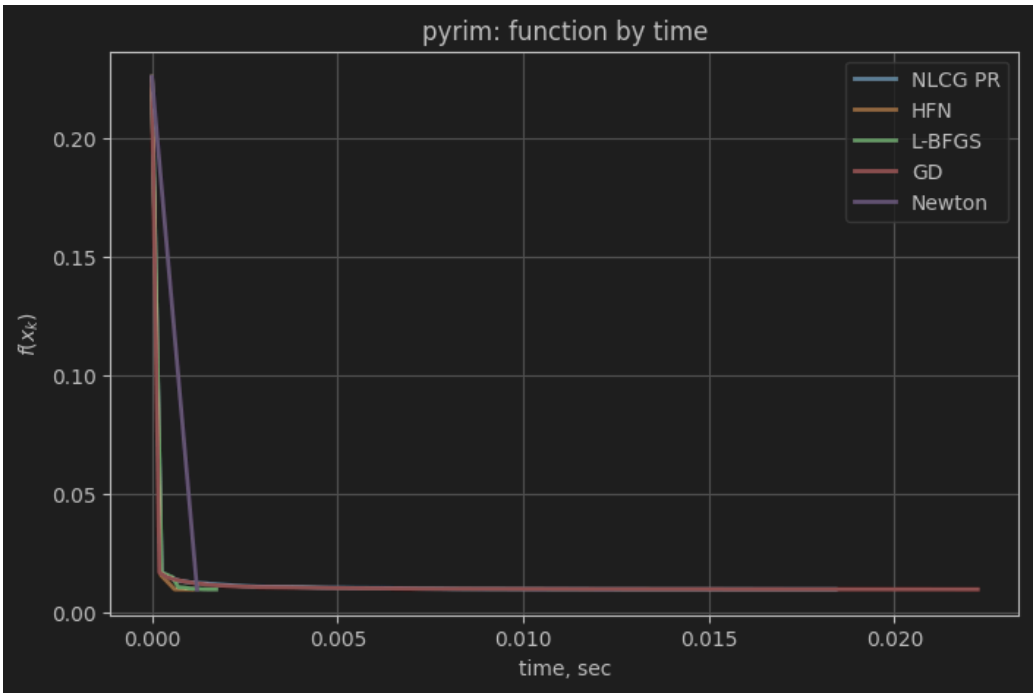


Рис. 21. `pyrim`: значение функции от времени.

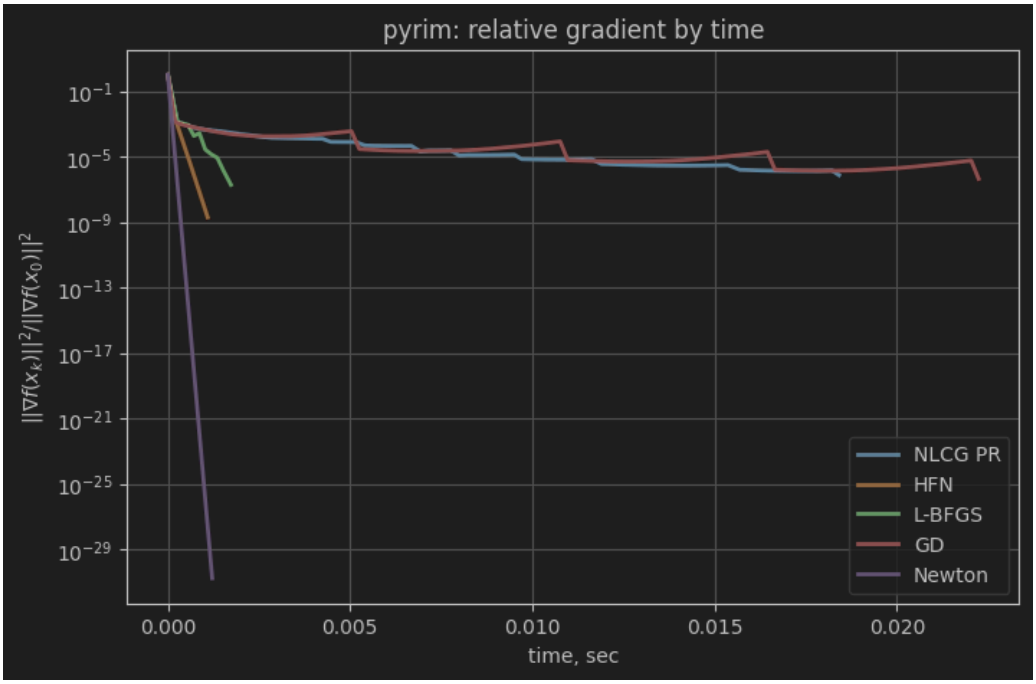


Рис. 22. `pyrim`: относительный квадрат нормы градиента от времени.

На `pyrim` картина похожа: полный Newton быстро достигает очень малой нормы градиента, а HFN и L-BFGS заметно быстрее GD. По графику функции все методы быстро выходят к близким значениям, но по норме градиента различия между методами заметнее.

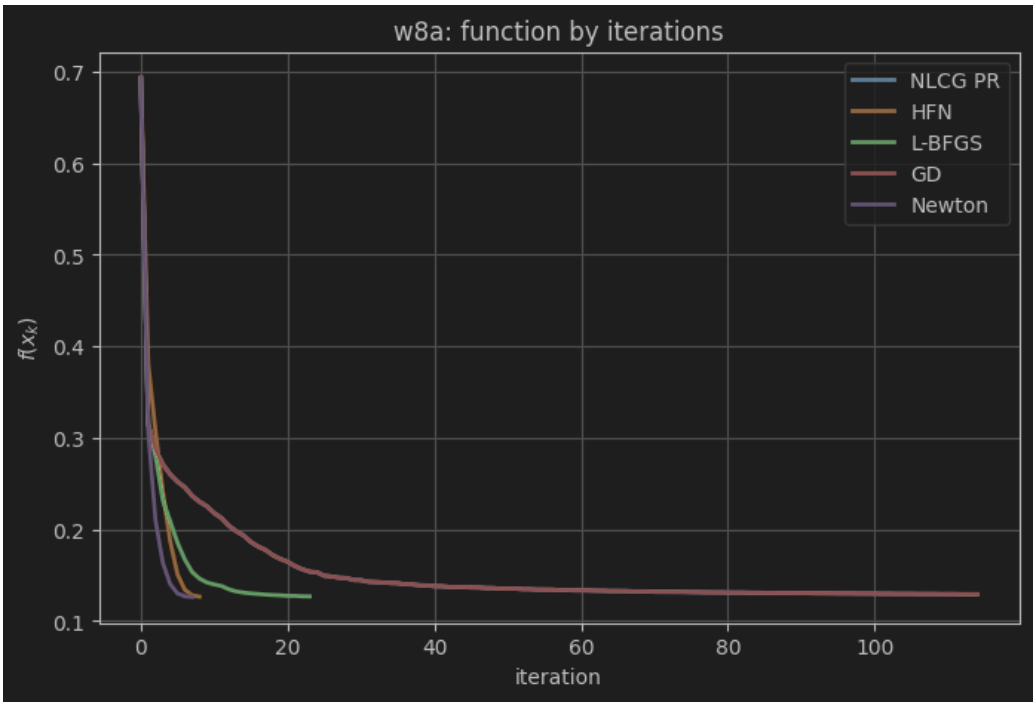


Рис. 23. w8a: значение функции от номера итерации.

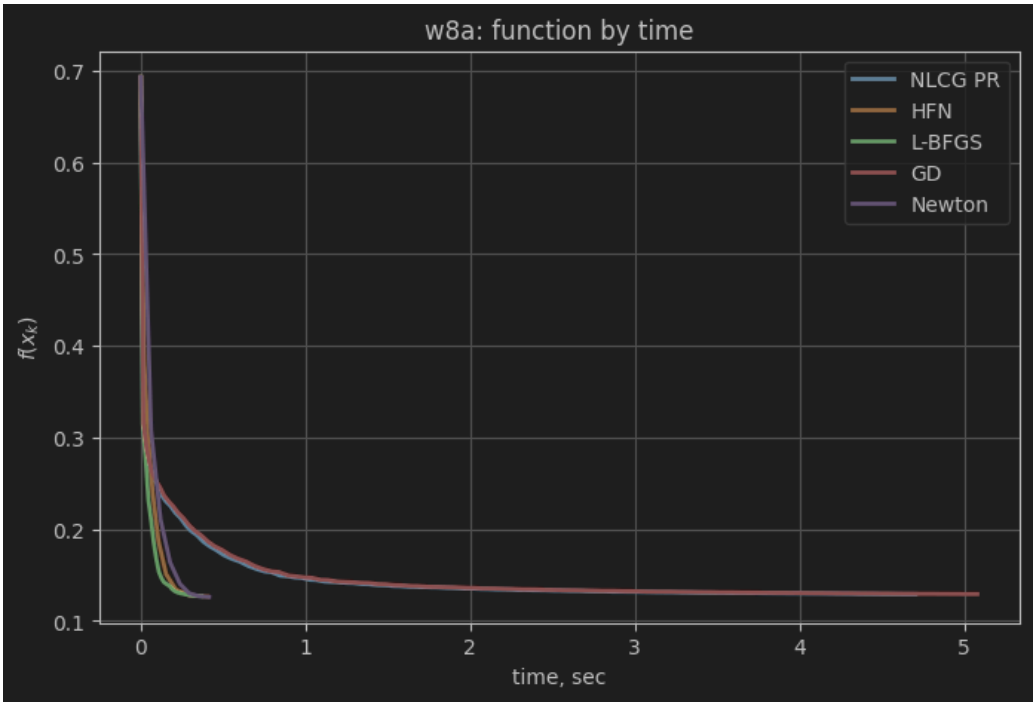


Рис. 24. w8a: значение функции от времени.

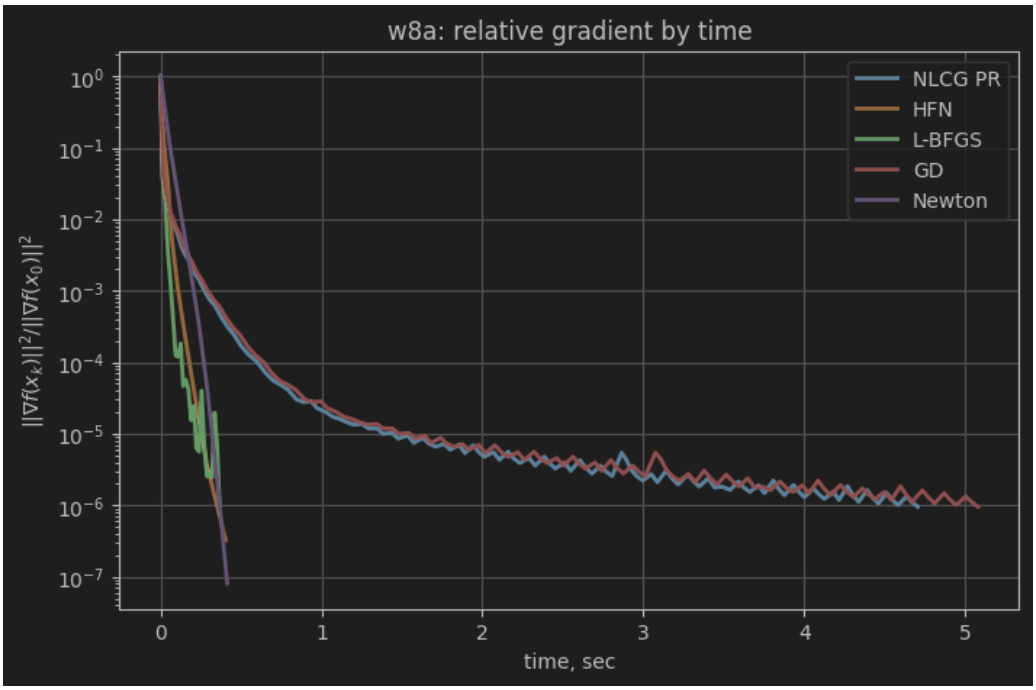


Рис. 25. `w8a`: относительный квадрат нормы градиента от времени.

На `w8a` GD и NLCG PR сходятся заметно дольше, чем HFN, L-BFGS и Newton. По времени HFN, L-BFGS и Newton оказываются близкими, при этом L-BFGS и HFN достигают малой невязки существенно быстрее, чем GD.

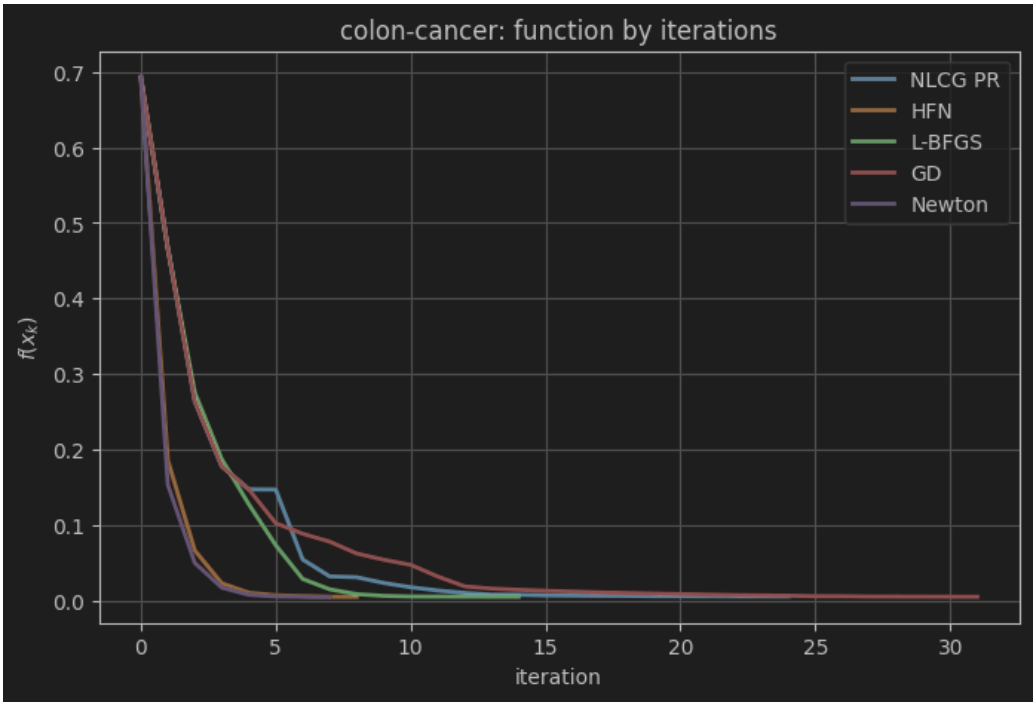


Рис. 26. `colon-cancer`: значение функции от номера итерации.

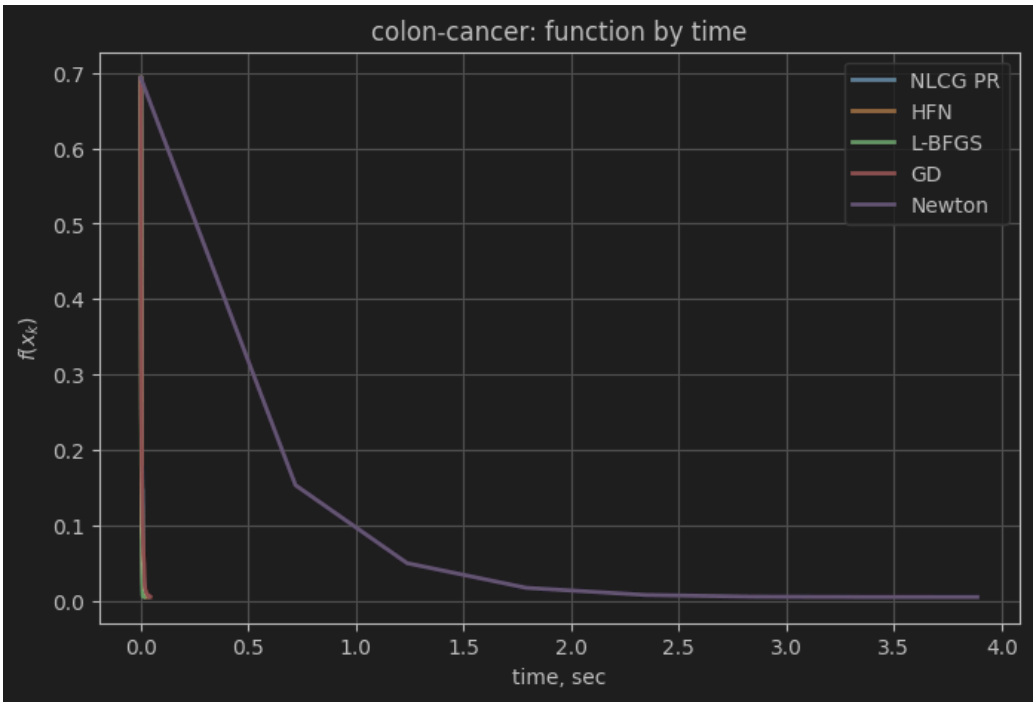


Рис. 27. `colon-cancer`: значение функции от времени.

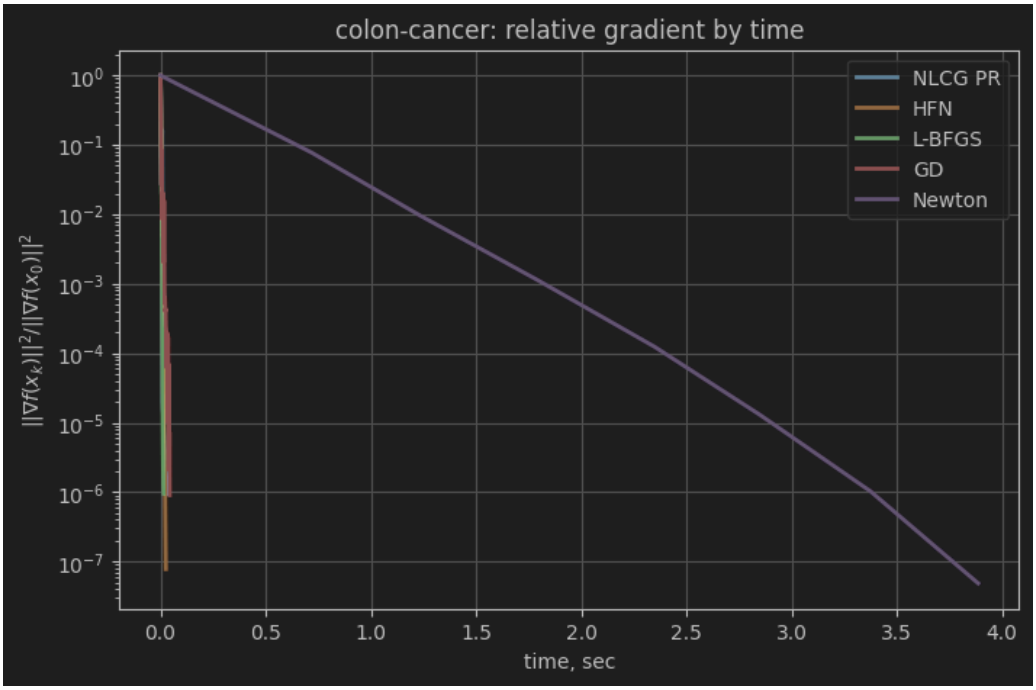


Рис. 28. `colon-cancer`: относительный квадрат нормы градиента от времени.

На `colon-cancer` полный Newton требует мало внешних итераций, но по реальному времени оказывается самым дорогим. HFN и L-BFGS сходятся значительно быстрее по времени, а GD снова проигрывает по числу итераций.

Ответы на вопросы

1. Меньше всего внешних итераций обычно делают Newton и HFN. Однако лидер по итерациям не всегда является лидером по времени: на `colon-cancer` полный Newton медленный в секундах, хотя по итерациям выглядит хорошо. Это связано с тем, что

одна итерация Newton намного дороже: нужно явно строить и факторизовать плотный гессиан, что особенно заметно при большой размерности n.

2. На высоких данных, например `w8a`, продвинутые методы HFN, L-BFGS и Newton заметно быстрее GD и NLCG PR. На широком датасете `colon-cancer`, где n велико, полный Newton становится дорогим по времени, а HFN и L-BFGS выглядят устойчивее. Причина в том, что L-BFGS хранит только небольшую историю векторов, а HFN использует произведения гессиана на вектор, тогда как полный Newton работает с большой плотной матрицей.

3. По графикам видно, что GD почти везде проигрывает продвинутым методам по скорости уменьшения нормы градиента. Это ожидаемо, потому что GD использует только направление антиградиента и не учитывает кривизну задачи. Полный Newton очень эффективен на малых задачах `housing_scale` и `pyrim`, но на `colon-cancer` проигрывает по реальному времени из-за высокой стоимости работы с гессианом. Поэтому на более широких задачах практичнее использовать HFN или L-BFGS.

Выводы

По результатам эксперимента HFN и L-BFGS оказались наиболее устойчивыми методами среди рассмотренных: они быстро сходятся на всех датасетах и не требуют явного построения плотного гессиана. Полный Newton дает очень быструю сходимость по итерациям, но может проигрывать по времени на задачах с большой размерностью. GD заметно уступает продвинутым методам, особенно на больших данных.

Эксперимент 2.5: Микропрофилирование вычислительных затрат

Постановка задачи

В эксперименте требуется для трёх продвинутых методов оптимизации

- NLCG PR,
- HFN,
- L-BFGS,

измерить, **куда уходит время внутри одной итерации** при оптимизации ML-оракулов (Ridge для регрессии и Probit для классификации) на датасетах.

Согласно методичке, время одной итерации нужно разложить на 3 компоненты:

- **Оракул:** вычисление ∇f (для всех методов) и $\nabla^2 f \cdot v$ (для HFN).
- **Линейная алгебра алгоритма:** внутренний цикл CG в HFN, двухцикловая рекурсия L-BFGS и т.п.
- **Линейный поиск:** вызовы функций/градиентов внутри проверки условий Вульфа.

Коэффициент регуляризации выбран стандартно: $\lambda = \frac{1}{m}$.

Используемые данные и модели

Использовались следующие датасеты и соответствующие задачи:

Датасет	**Задача**	m	n
housing_scale	регрессия (Ridge)	506	13
pyrim	регрессия (Ridge)	74	27
w8a	классификация (Probit)	49749	300
colon-cancer	классификация (Probit)	62	2000

Таблица 3. Сводка по датасетам для эксперимента 2.5. Здесь m — число объектов, n — число признаков.

Результаты

Ниже столбчатые диаграммы для каждого датасета: время итерации разбито на доли **oracle / linear algebra / line search**.

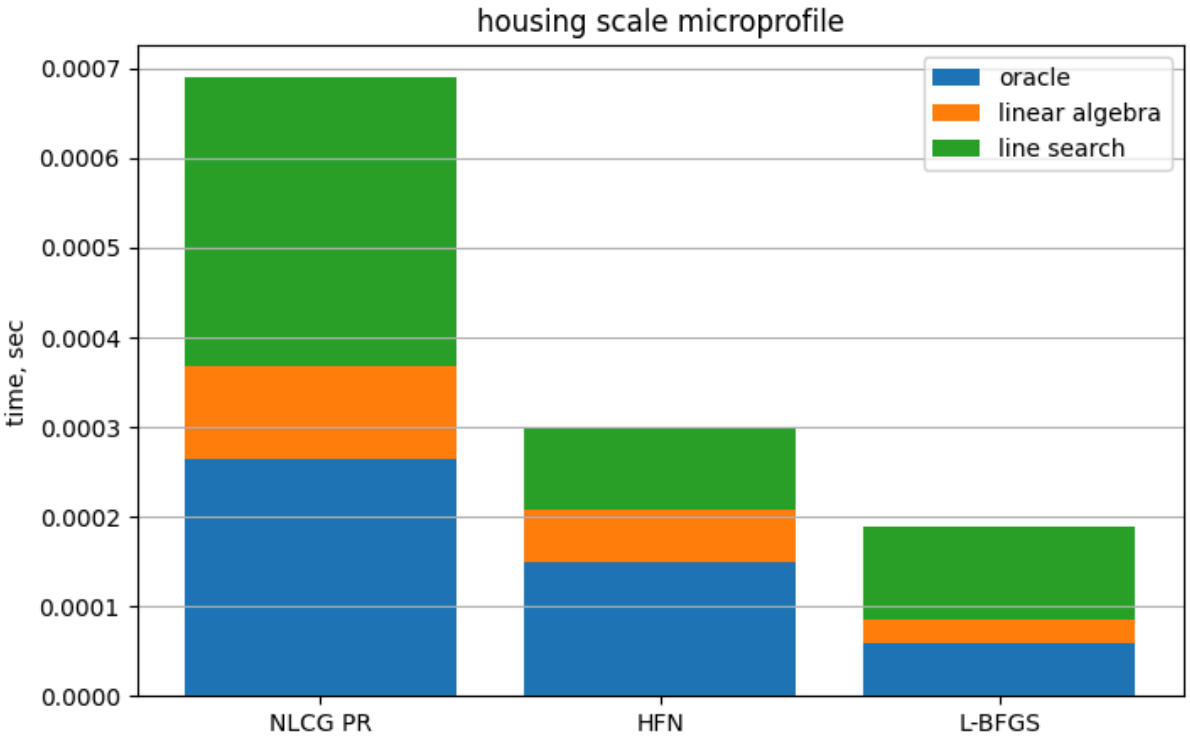


Рис. 29. Эксп. 2.5: housing_scale.

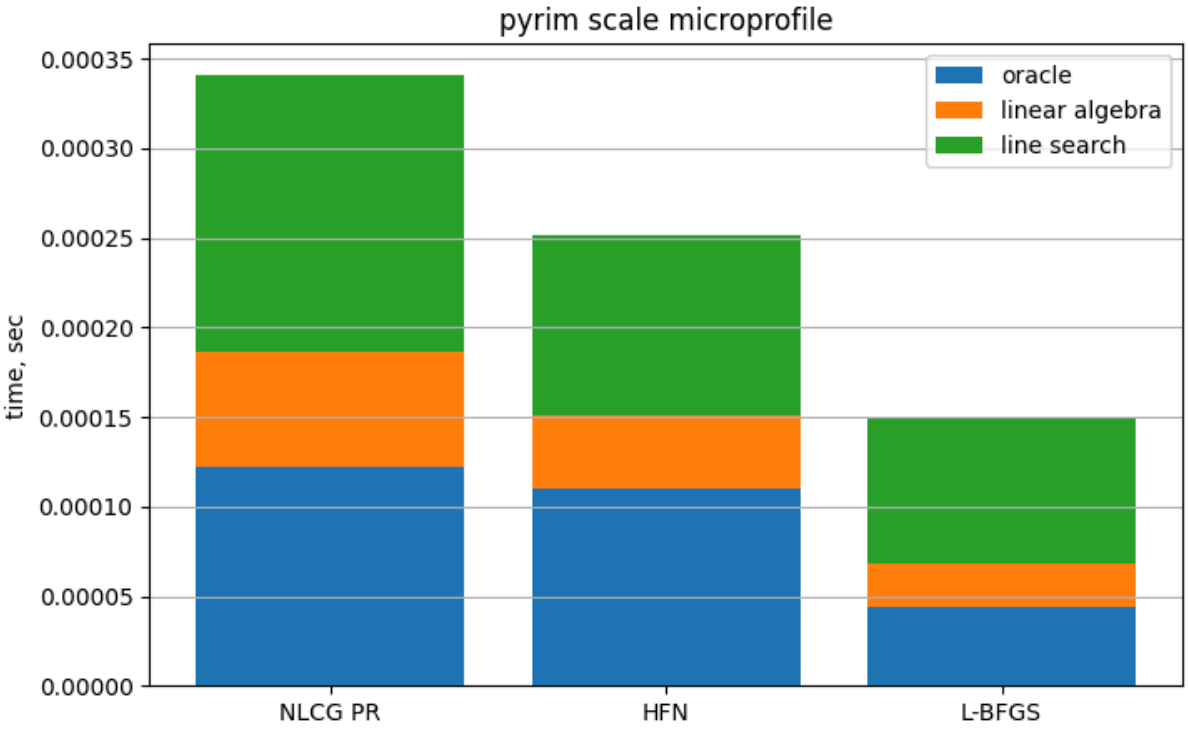


Рис. 30. Эксп. 2.5: pyrim.

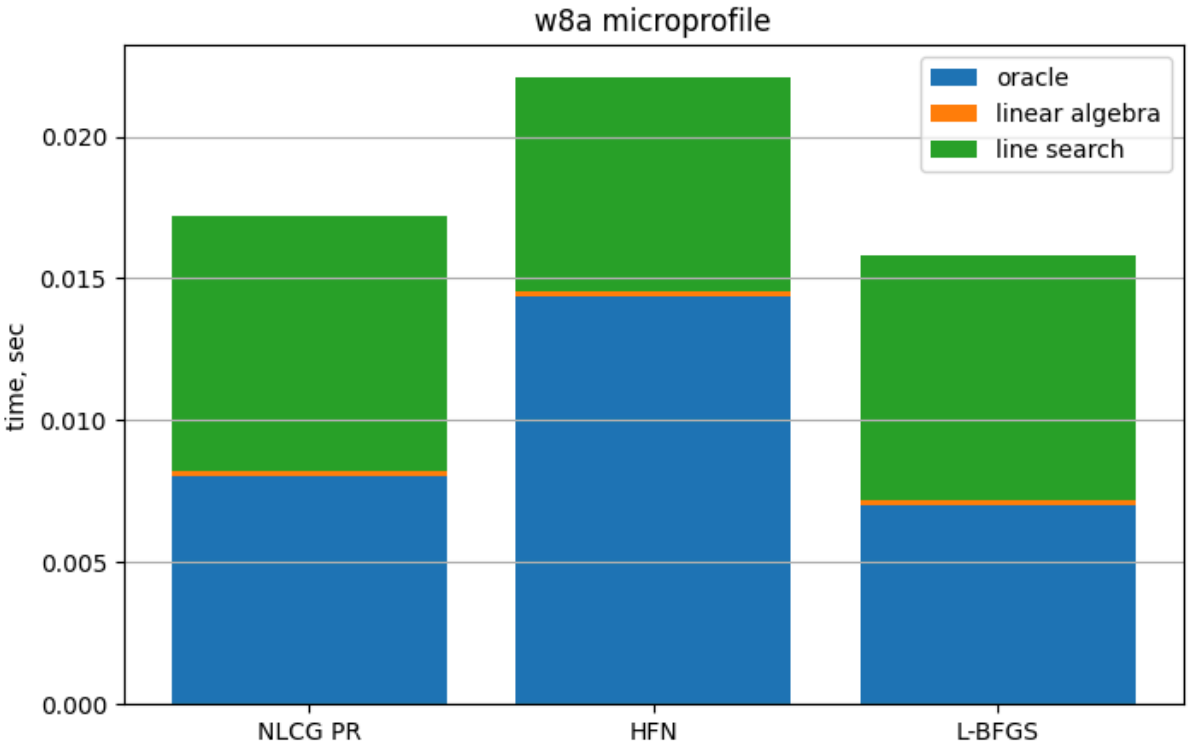


Рис. 31. Эксп. 2.5: w8a.

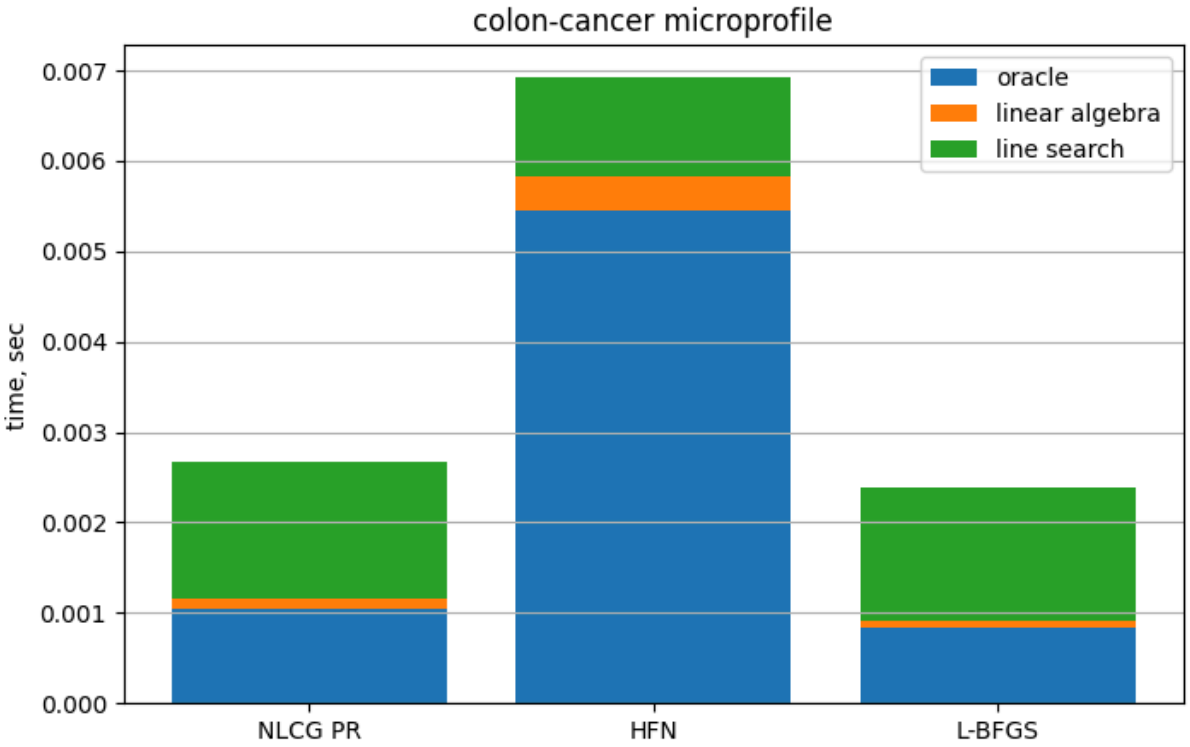


Рис. 32. Эксп. 2.5: colon-cancer ($n \ll m$).

Ответы на вопросы

1. **У какого метода линейный поиск «съедает» самую большую долю времени?**

По всем датасетам максимальная доля времени на линейный поиск наблюдается у **L-BFGS**. На `w8a` доли линейного поиска у L-BFGS и NLCG PR оказываются близкими (примерно 49% и 47%), но L-BFGS всё равно остаётся лидером по доле.

Почему так происходит:

- В L-BFGS часть итерации oracle — является дешёвой векторной процедурой, поэтому относительный вклад накладных расходов линейного поиска становится большим.
- При Wolfe line search подбирается α, и для проверки условий выполняются дополнительные вызовы $f(x + \alpha d)$ и $\nabla f(x + \alpha d) \cdot d$. В нашем профилировании время этих oracle-вызовов учитывается внутри компоненты **line search**, что также увеличивает её долю.

2. **Почему на задачах с $n \ll m$ усечённый Ньютон может проигрывать L-BFGS по времени?**

`colon-cancer` ($m = 62$, $n = 2000$) является примером задачи, где $n \ll m$. На нём HFN в наших измерениях имеет заметно более дорогую итерацию, чем L-BFGS.

Причины:

- В HFN ньютоновское направление ищется внутренним методом CG. Каждая итерация CG требует операций вида $\nabla^2 f \cdot v$, что становится доминирующей частью времени.
- При $n \ll m$ в реализации HFN направление находится в n-мерном пространстве через CG, что является вызовом оракула второго порядка, т. е. $\nabla^2 f \cdot v$.

- В отличие от HFN, L-BFGS не использует $\nabla^2 f \cdot v$ вовсе и аппроксимирует кривизну по небольшой истории шагов. Поэтому при $n \ll m$ L-BFGS часто выигрывает по времени итерации, даже если линейный поиск занимает большую долю.

Выводы

По результатам микропрофилирования можно выделить две устойчивые закономерности:

- **Линейный поиск** чаще всего является самым дорогим относительным компонентом для L-BFGS И NLCG PR, поскольку внутренняя линейная алгебра этих методов сравнительно дешева.
- **Усечённый Ньютон** может проигрывать на задачах с $n \ll m$ из-за необходимости выполнять множества сложных вычислительных операций внутри CG; на `colon-cancer` это проявляется как доминирование компоненты **oracle** в общей стоимости итерации.

Эксперимент 2.6: Оптимизационная точность против качества предсказания

Постановка задачи

В прикладных задачах машинного обучения целевая функция на обучающей выборке $f_{\text{train}}(x)$ не является конечным критерием качества. Нас интересует качество на тестовой выборке (Accuracy для классификации и ошибки MSE/MAE для регрессии). Поэтому важно понять, **после какого уровня оптимизационной точности дальнейшая оптимизация становится бесполезной**.

- берём один метод оптимизации (в нашем случае L-BFGS),
- запускаем его на train-части с высокой точностью $\varepsilon = 10^-8$,
- на каждой итерации k фиксируем:
  - $f_{\text{train}}(x_k)$,
  - $\|\nabla f(x_k)\|_2$,
  - тестовую метрику качества.

Далее строим график с двумя осями Y:

- слева (логарифмически): $f_{\text{train}}(x_k)$ и $\|\nabla f(x_k)\|_2$,
- справа (линейно для Accuracy и логарифмически для MSE/MAE): тестовая метрика.

Данные, модели и настройки

Используются те же ML-оракулы, что и в предыдущих экспериментах:

- Ridge-регрессия для датасетов `housing_scale`, `pyrim`,
- Probit-классификация для датасетов `w8a`, `colon-cancer`.

Данные разделяются на train/test в пропорции 80/20. Для воспроизводимости используется `seed = 67`.

Коэффициент регуляризации выбран стандартно (кроме графиков пункта 3): $\lambda = \frac{1}{m}$, где m — число объектов на train.

Алгоритм оптимизации: L-BFGS и Wolfe line search. Критерий остановки соответствует относительной точности по квадрату нормы градиента:

$$\|\nabla f(x_k)\|_2^2 \leq \varepsilon * \|\nabla f(x_0)\|_2^2.$$

Результаты

Полученные графики «оптимизация vs качество» на тесте приведены ниже:

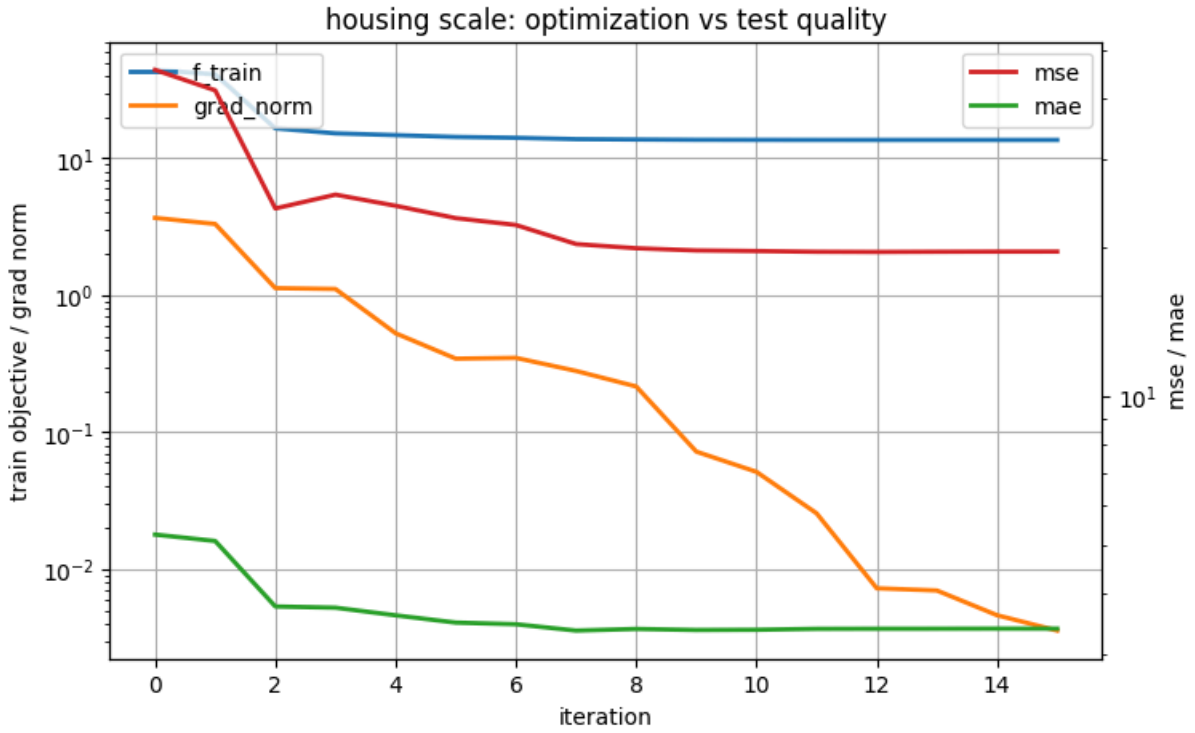


Рис. 33. Эксп. 2.6: `housing_scale` (Ridge)

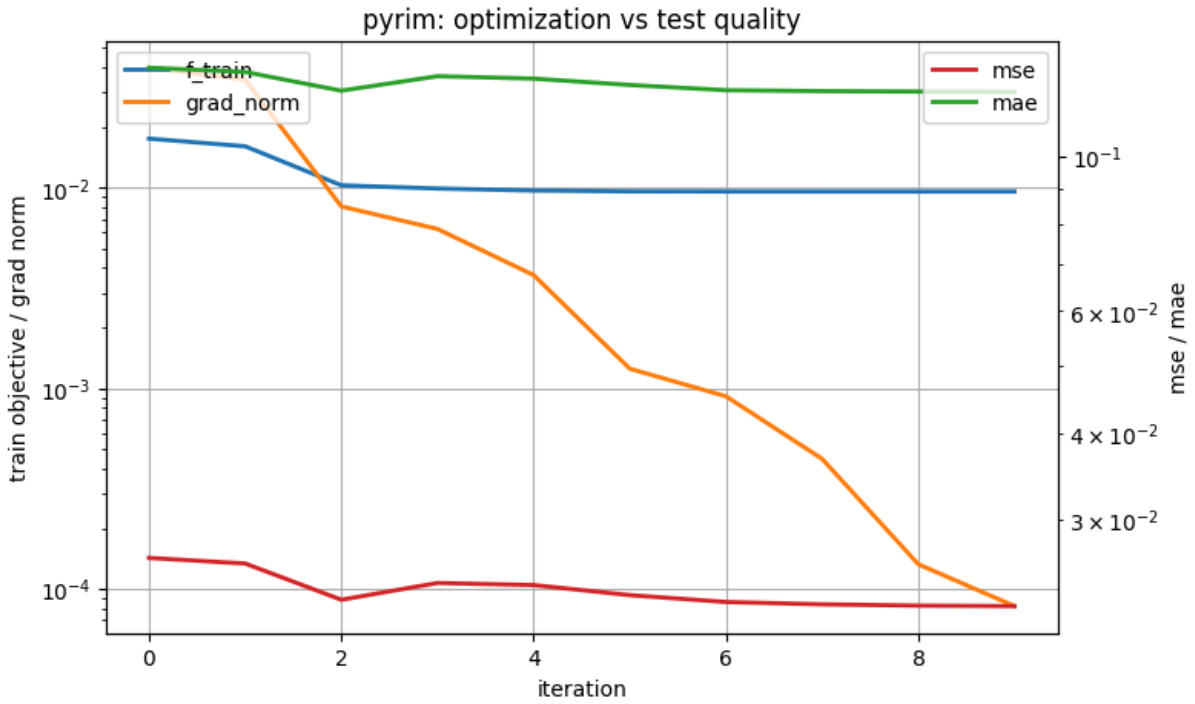


Рис. 34. Эксп. 2.6: `pyrim` (Ridge)

Рис. 35. Эксп. 2.6: `w8a` (Probit)

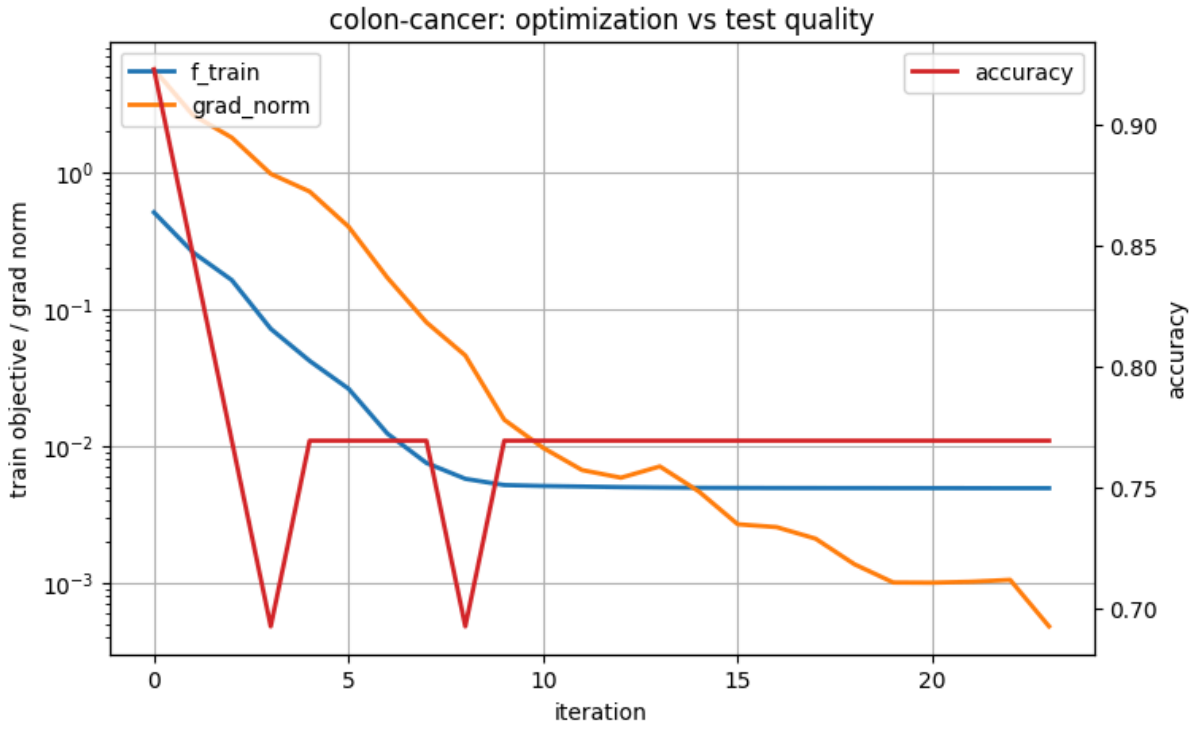


Рис. 36. Эксп. 2.6: `colon-cancer` (Probit)

Итоговые оценки (получены тем же кодом, что строит графики):

Замечание по `colon-cancer`: тестовая выборка очень мала, поэтому добавление новых объектов сильно влияет на модель, из-за чего Accuracy меняется ступенчато.

Ответы на вопросы

1. **На какой итерации (или при какой норме градиента) метрика качества на тесте перестает значимо улучшаться? Совпадает ли этот момент с визуальной сходимостью функции потерь?**

По графикам видно, что метрика на тесте выходит на плато заметно раньше, чем полностью сходится $\|\nabla f(x_k)\|_2$:

- `w8a`: Accuracy практически перестаёт расти после $k \approx 16$ (при $\|\nabla f\|_2 \approx 2.3 * 10^{-3}$), тогда как $\|\nabla f\|_2$ продолжает падать ещё десятки итераций.
- `colon-cancer`: после $k \approx 9$ Accuracy стабилизируется около 0.769, а оптимизация дальше лишь уменьшает f_{train} и норму градиента.
- Для регрессии (`housing_scale`, `pyrim`) MSE/MAE улучшаются в первые несколько итераций, после чего изменения становятся малозаметны.

Следовательно, визуальная сходимость функции потерь (и нормы градиента) не совпадает с моментом, когда перестаёт улучшаться тестовое качество.

2. **Какую долю общего времени работы алгоритма занимает бесполезная оптимизация? Какое ε стоит использовать для реальных задач?**

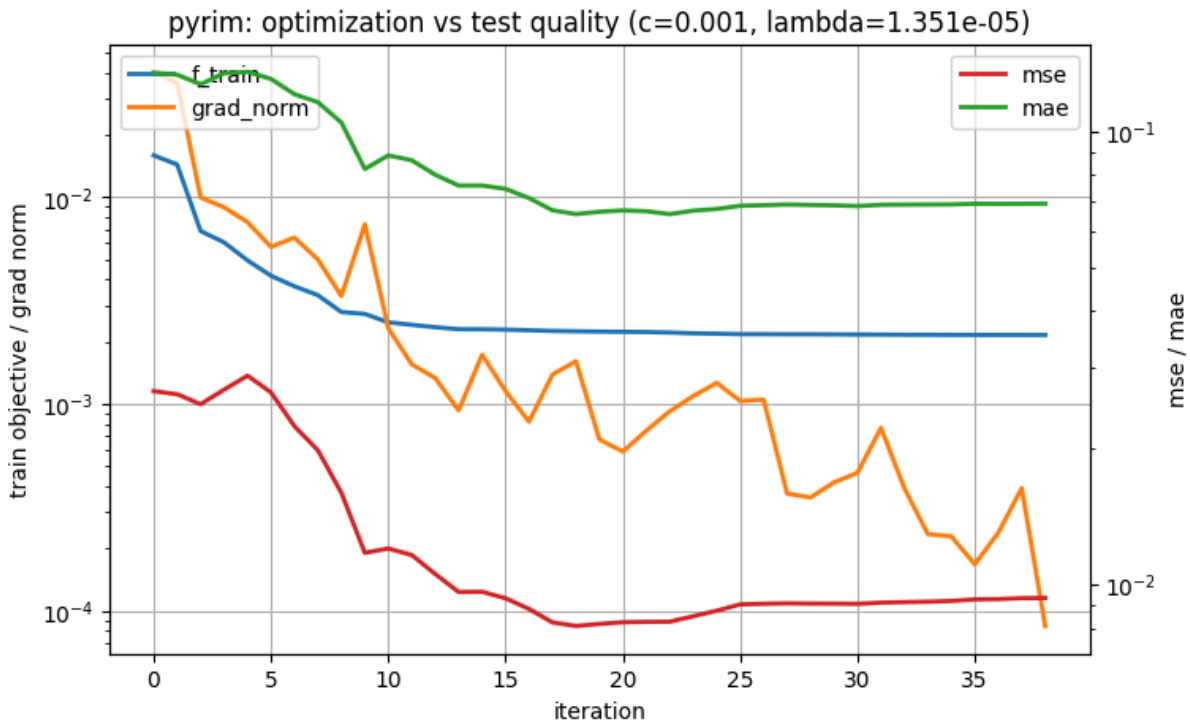
Из табл. видно, что при очень строгой точности $\varepsilon = 10^-8$ существенная часть времени уходит на оптимизацию после насыщения тестовой метрики:

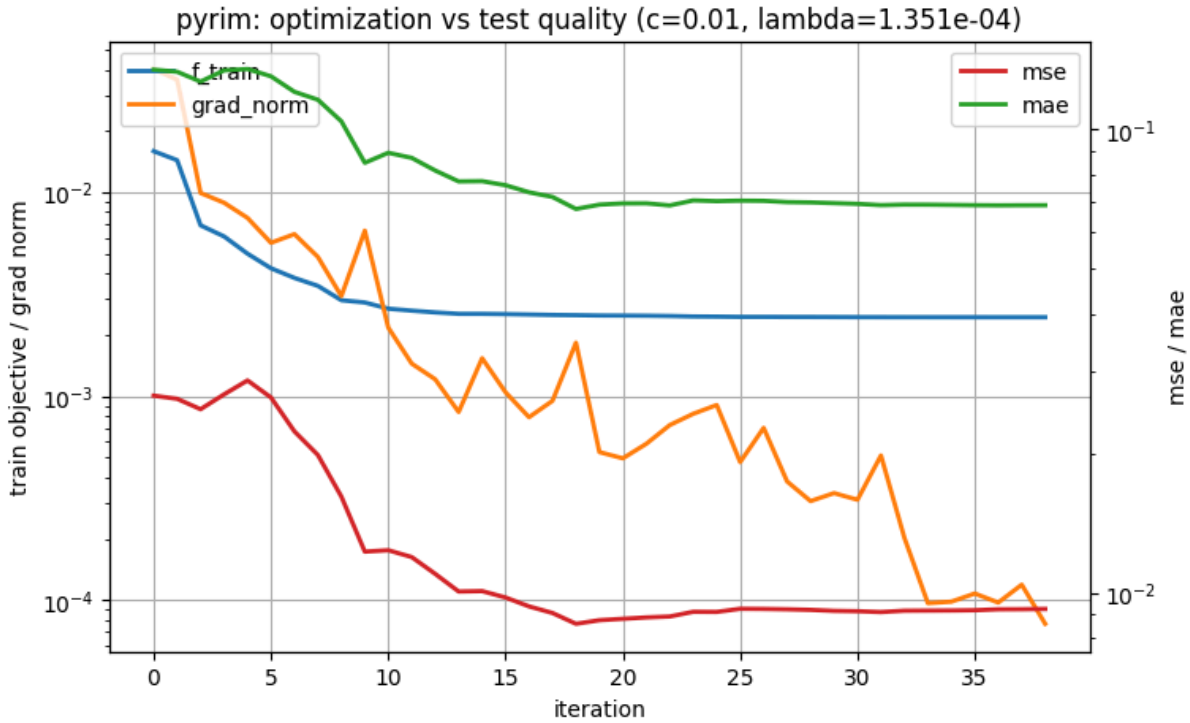
- `housing_scale`: около 45% времени;
- `pyrim`: около 40% времени.
- `w8a`: около 65% времени;
- `colon-cancer`: около 55% времени;

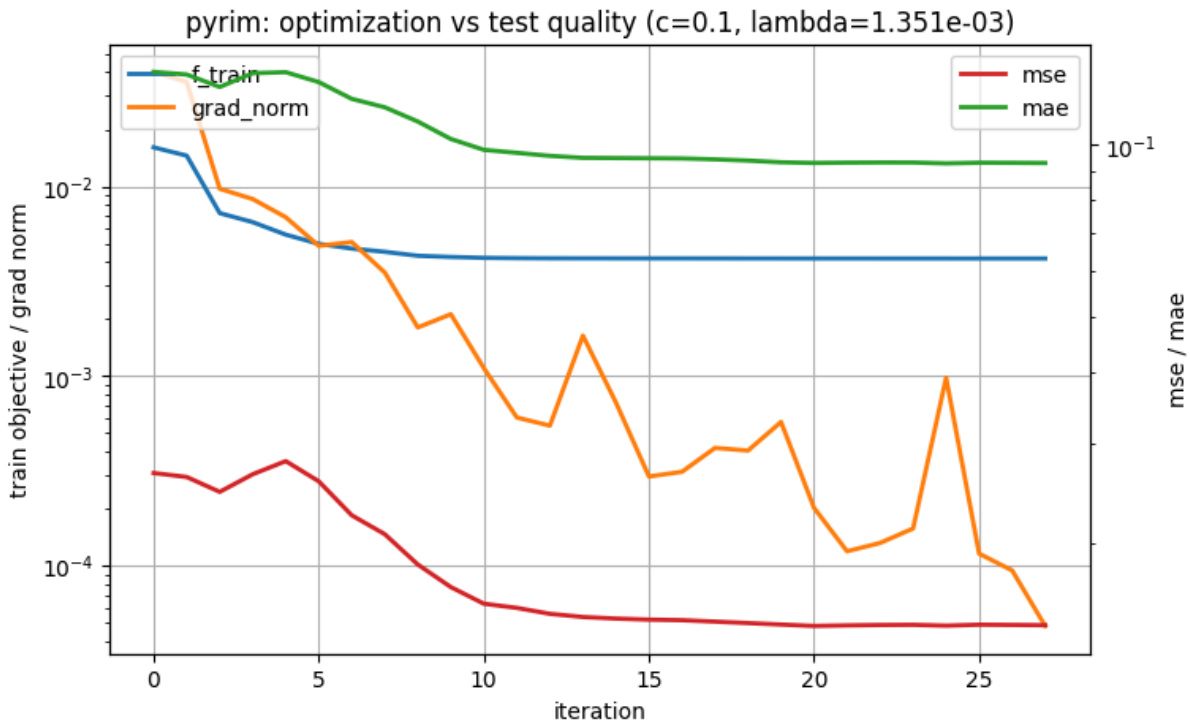
То есть на практике имеет смысл останавливать оптимизацию существенно раньше. Если ориентироваться на относительный критерий $\frac{\|\nabla f(x_k)\|_2^2}{\|\nabla f(x_0)\|_2^2}$, то уровень ε, соответствующий выходу тестовой метрики на плато, это примерно $10^{-4} - 10^{-5}$.

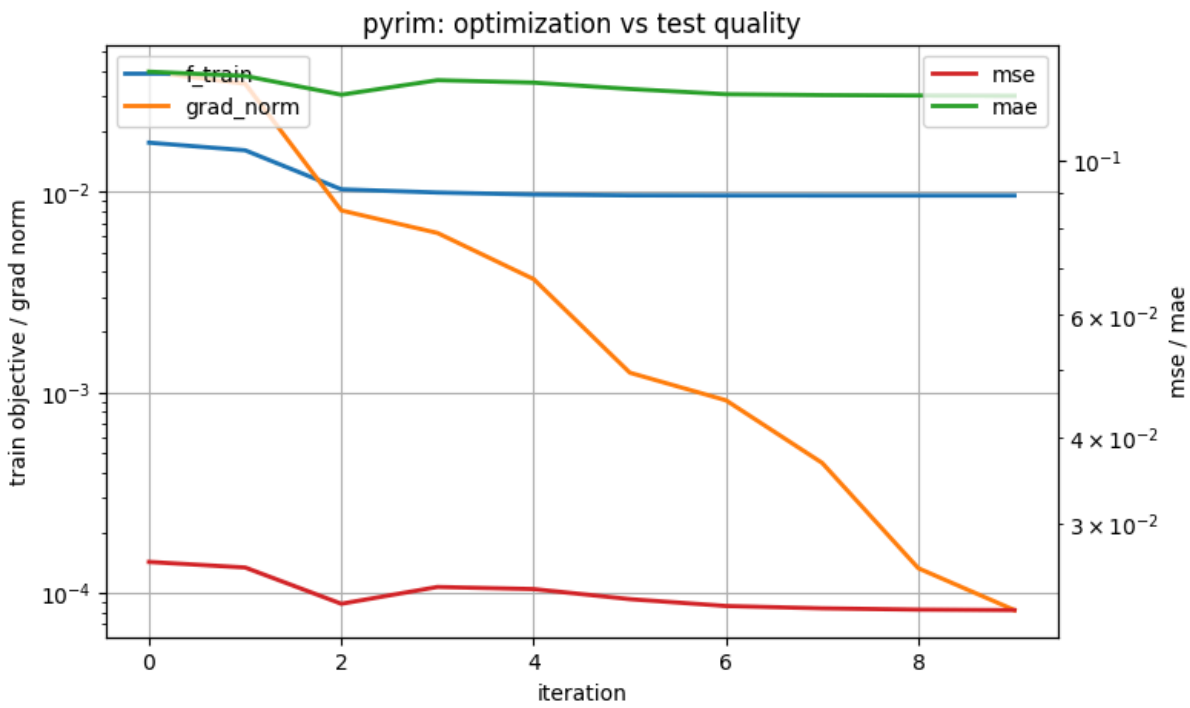
3. **Как меняется точка насыщения при изменении параметра λ? Становится ли разрыв между обучением и тестом больше при очень маленьких λ?**

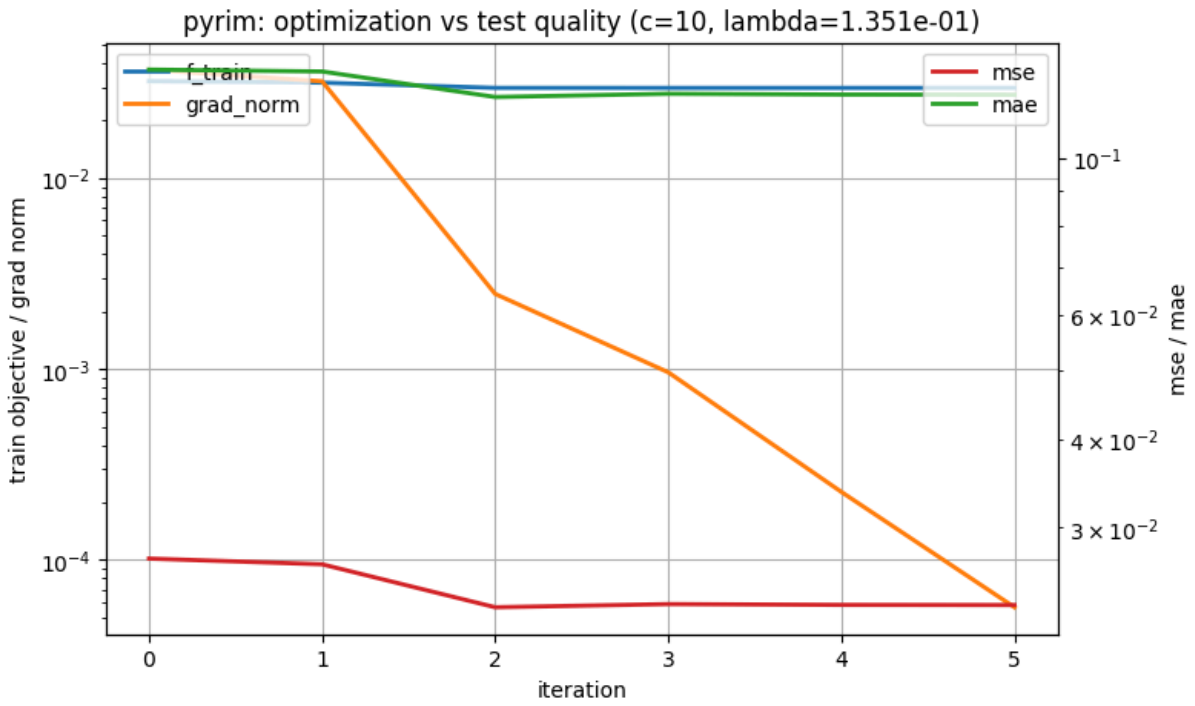
На датасете housing_scale изменения почти не видны, поэтому лучше рассмотрим pyrim. По полученным графикам видно, что при увеличении c метод сходится быстрее. При малых λ целевая функция оптимизируется дольше, однако тестовые метрики (MSE/MAE) выходят на плато относительно рано, и дальнейшее вычисление $f_{\text{train}}(x)$ и $\|\nabla f(x)\|_2$ даёт минимальный выигрыш в качестве на тесте. Поэтому с увеличением λ точка насыщения сдвигается к первым итерациям.


pyrim: optimization vs test quality (c=0.001, lambda=1.351e-05)
f_train
grad_norm
mse
mae
train objective / grad norm
mse / mae
iteration


pyrim: optimization vs test quality (c=0.01, lambda=1.351e-04)
f_train
grad_norm
mse
mae
train objective / grad norm
mse / mae
iteration


pyrim: optimization vs test quality (c=0.1, lambda=1.351e-03)
f_train
grad_norm
mse
mae
train objective / grad norm
mse / mae
iteration


pyrim: optimization vs test quality
f_train
grad_norm
mse
mae
train objective / grad norm
mse / mae
iteration



В классификации изменения λ повлияли только на количество итераций.

Выводы

Эксперимент показывает, что оптимизация до очень строгих значений ε не гарантирует улучшения качества на тесте. Для реальных задач разумнее выбирать ε на уровне, который соответствует плато тестовой метрики (в наших экспериментах это примерно 10^{-4}–10^{-5}), чтобы не тратить значительную долю времени на бесполезное уменьшение $f_{\text{train}}(x)$ и $\|\nabla f(x)\|_2$.

Эксперимент 2.7 (Индивидуальный трек): Потеря ортогональности в нелинейном методе сопряжённых градиентов и формула Hager–Zhang

Цель и постановка

Исследуется эффект потери ортогональности в нелинейном методе сопряжённых градиентов и влияние рестарта Пауэлла. Сравниваются четыре конфигурации: PR без рестарта Пауэлла, PR с рестартом Пауэлла, HZ без рестарта Пауэлла и HZ с рестартом Пауэлла.

Для оценки потери «памяти» метода на каждой итерации анализируется величина

$$\cos(\theta_k) = \frac{|g_k^T g_{k-1}|}{\sqrt{(g_k^T g_k)(g_{k-1}^T g_{k-1})}}.$$

Дополнительно сравнивается скорость сходимости методов по числу итераций и по времени.

Настройки

Эксперимент проводился на датасете w8a для задачи классификации. Начальное приближение: $x_0 = 0$. Использовался нелинейный метод сопряжённых градиентов с линейным поиском по условиям Вульфа, tolerance = 1e-6, max_iter = 500.

Рассматривались следующие конфигурации:

- PR no Powell
- PR Powell
- HZ no Powell
- HZ Powell

В вариантах с рестартом включался критерий Пауэлла; в вариантах без рестарта история не сбрасывалась. Для анализа частоты рестартов фиксировались число срабатываний и их доля от общего числа итераций.

Данные

Использовался датасет w8a; решалась задача бинарной классификации с ML-оракулом. Предобработка данных соответствовала базовой реализации оракула для данного набора данных.

Результаты

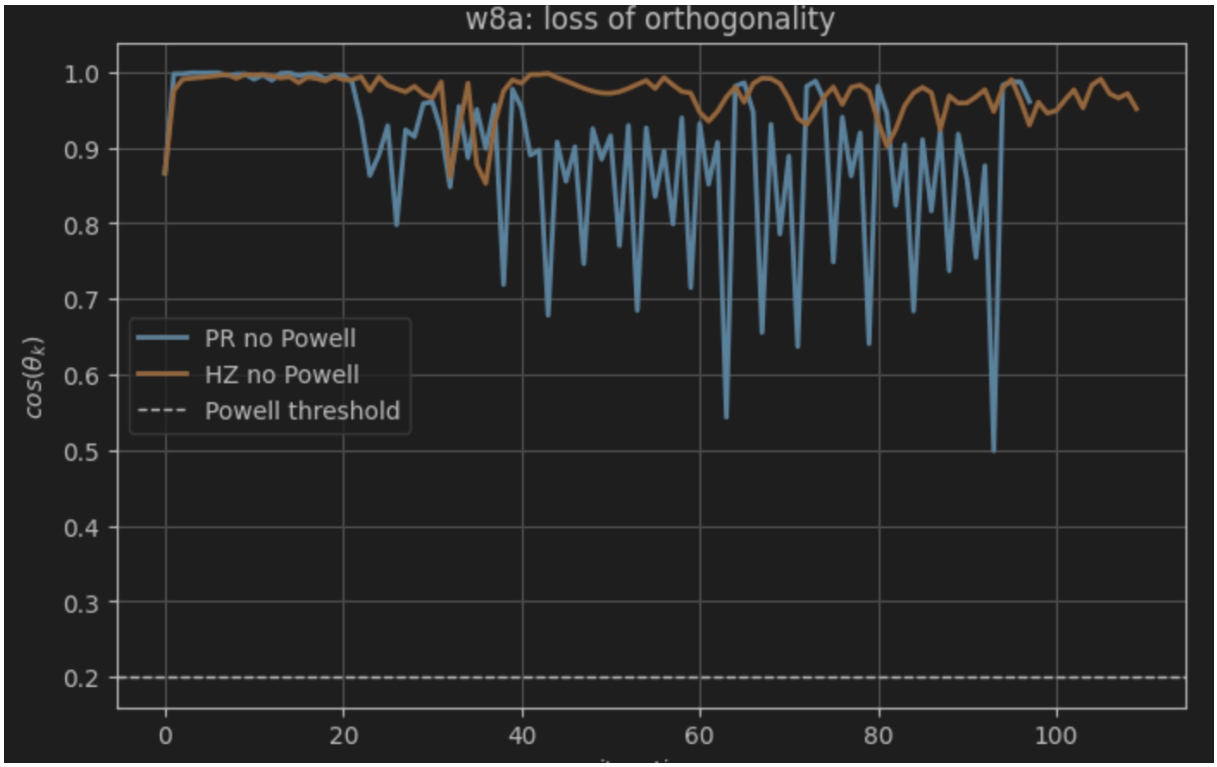


Рис. 37. Значение $\cos(\theta_k)$ для PR и HZ без рестартов«

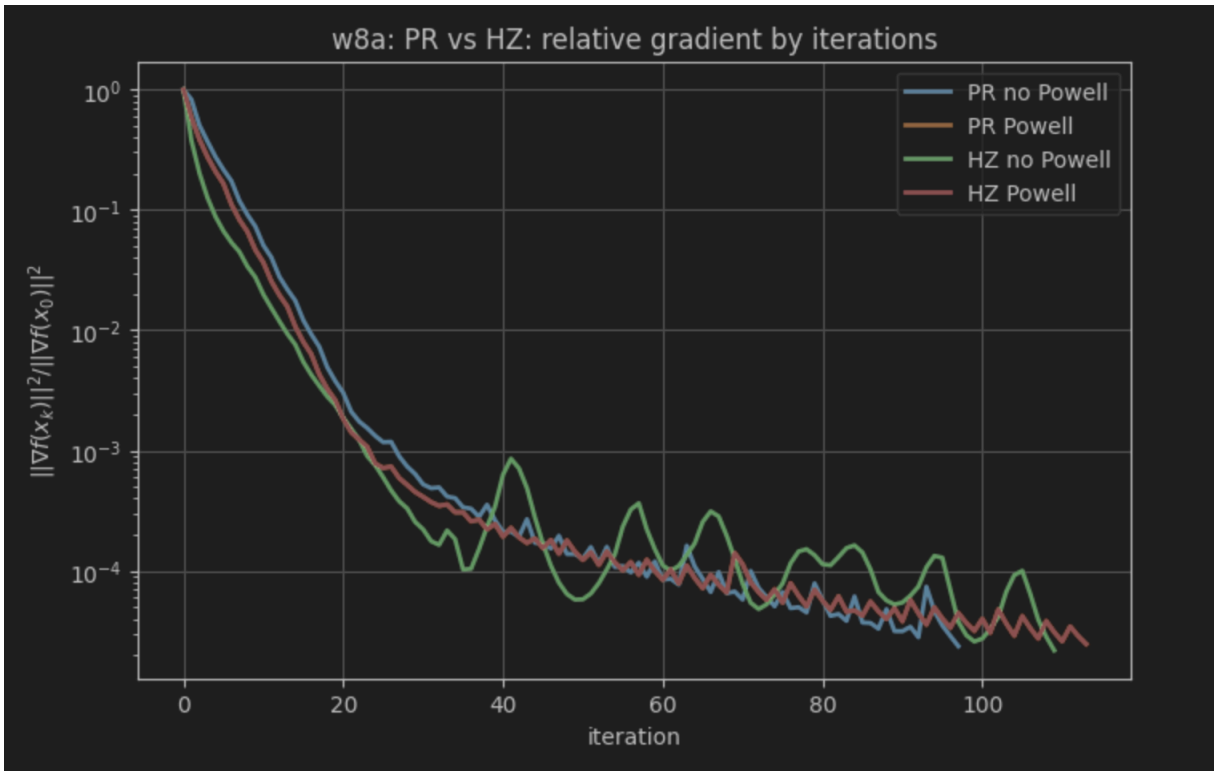


Рис. 38. Сходимость по итерациям для PR/HZ с рестартом Пауэлла и без

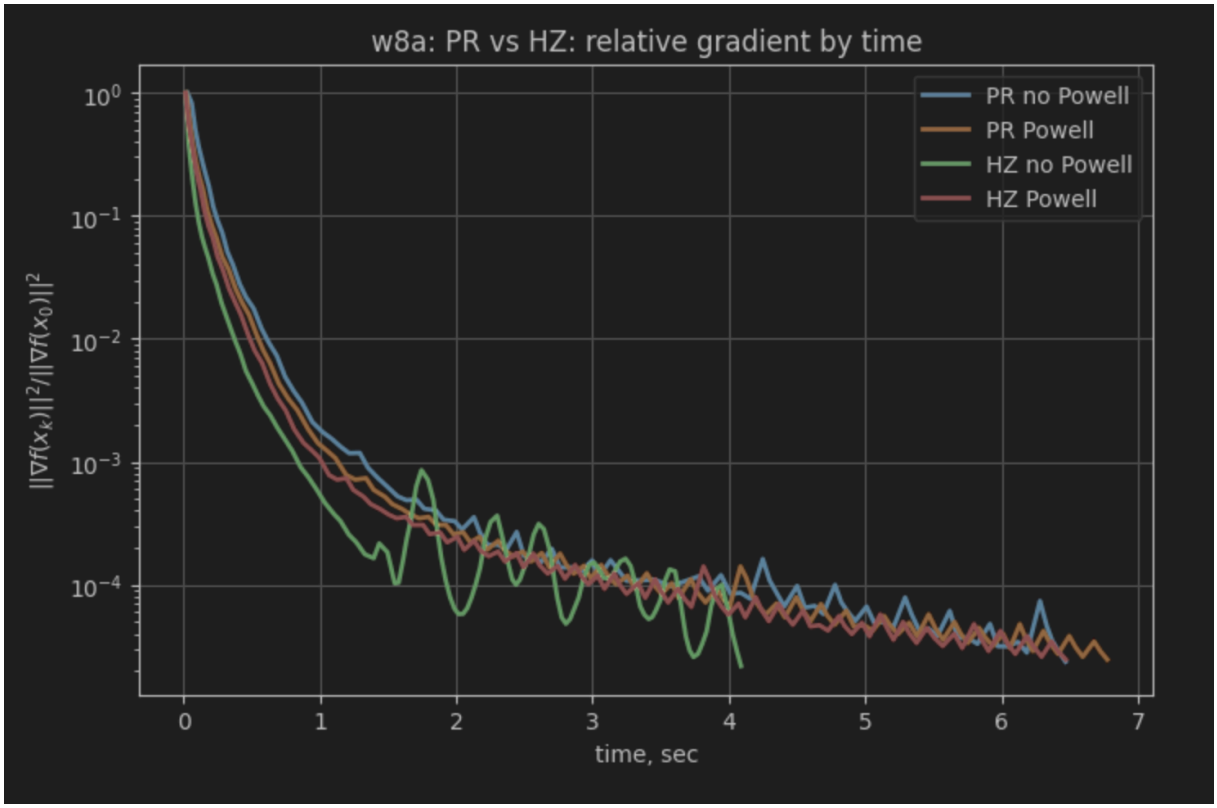


Рис. 39. Сходимость по времени для PR/HZ с рестартом Пауэлла и без

Анализ результатов

1. График $\cos(\theta_k)$ показывает, что потеря ортогональности возникает очень быстро: уже в первые итерации косинус резко уходит от нуля и становится близким к 1. Это

означает, что соседние градиенты перестают быть почти ортогональными, и метод теряет «память» о предыдущих направлениях. Для PR без рестартов величина $\cos(\theta_k)$ заметно колеблется, но остаётся большой; для HZ без рестартов косинус почти всё время держится в диапазоне около $0.95 - 1.0$. Следовательно, на данном датасете деградация ортогональности наблюдается у обоих методов и происходит практически сразу.

2. Поскольку порог рестарта Пауэлла равен 0.2, а на графике $\cos(\theta_k)$ почти на всех итерациях значительно больше этого значения, критерий рестарта в методе PR должен срабатывать очень часто, практически на каждом шаге после первых итераций. То есть без перезапуска метод быстро теряет полезную историю, а рестарт Пауэлла в этой задаче оказывается действительно востребованным.

3. По графикам сходимости видно, что HZ в целом не хуже PR, а без рестарта даже выглядит предпочтительнее: кривая HZ no Powell убывает быстрее в начале и достигает малых значений нормы градиента за меньшее время. Однако явного большого отрыва HZ от PR во всех режимах нет: при включении рестарта Пауэлла кривые PR Powell и HZ Powell становятся очень близкими. Это означает, что основное улучшение даёт именно механизм рестарта, а не только выбор формулы для β_k.

При этом формула Hager–Zhang не устраняет потерю ортогональности полностью. По графику $\cos(\theta_k)$ видно, что у HZ без рестартов косинус также далёк от нуля и часто почти равен 1, то есть сильная деградация ортогональности сохраняется. Тем не менее, даже при такой деградации HZ даёт более устойчивое направление спуска и потому на практике сходится не хуже, а местами лучше PR.

Выводы

На датасете w8a эффект потери ортогональности выражен очень сильно: для PR и HZ без рестартов величина $\cos(\theta_k)$ почти сразу становится большой, поэтому метод быстро «забывает» исходную сопряжённость направлений. Критерий рестарта Пауэлла в таких условиях должен срабатывать очень часто и действительно стабилизирует поведение метода. Формула Hager–Zhang сама по себе не избавляет от деградации ортогональности, но по скорости сходимости выглядит как минимум не хуже классического PR, а без рестартов даже лучше. В целом на этой задаче наилучшее практическое поведение показывают варианты с рестартом Пауэлла, а HZ можно считать более устойчивой альтернативой PR.